

FPT SCHOOL OF BUSINESS & TECHNOLOGY

MASTER'S THESIS

A Modular Detection and Completion Framework for Occluded Vehicles in Automotive LiDAR

Student:

Ngo Vi Viet Anh	MSE13205

Advisor:
Dr. Doan Nhat Quang

Master's Thesis
September 15, 2026

Keywords

- Autonomous driving perception
- Automotive LiDAR
- Point cloud segmentation
- Vehicle detection
- Point cloud completion
- Occluded-vehicle completion
- Modular perception pipeline
- Auto-mined supervision
- Reference-free evaluation
- SemanticKITTI
- ShapeNet

Acknowledgements and Declaration of Authorship

Acknowledgements

I would like to express my sincere gratitude to my thesis supervisor, **Dr. Doan Nhat Quang**, for his guidance, constructive feedback, and support throughout the development of this research. His academic advice and technical insights have been essential to the conceptualization and refinement of the proposed framework.

I am also grateful to the faculty and staff at the **FPT School of Business & Technology (FSB)** for providing a structured academic environment and for the knowledge they have shared throughout the Master's programme. I also thank my colleagues and peers for their helpful discussions and feedback during the research process.

Finally, I wish to thank my family for their patience and support during the period of my graduate studies.

Declaration of Authorship

I, **Ngo Vi Viet Anh**, student ID **MSE13205**, hereby declare that this thesis, entitled *"A Modular Detection and Completion Framework for Occluded Vehicles in Automotive LiDAR"*, is the result of my own work carried out under the supervision of Dr. Doan Nhat Quang at the FPT School of Business & Technology.

I solemnly declare that:

- The research presented in this work, including the modular detection-and-completion framework, its training with zero additional manual annotation on SemanticKITTI and ShapeNet, and the evaluation of occluded-vehicle completion, is my own original work unless otherwise referenced.
- All experimental results and quantitative evaluations reported in Chapter 4 are authentic and have been obtained through testing on the specified datasets. No data fabrication or manipulation has taken place.
- All sources of information utilized in this thesis have been cited in accordance with standard academic referencing practices.
- This thesis has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

I take full responsibility for the content, accuracy, and academic integrity of this thesis.

Contents

1 Introduction **1**
- 1.1 Background and Rationale . . . 1
- 1.2 Completing Occluded Vehicles on Real LiDAR . . . 1
- 1.3 Research Objectives . . . 2
- 1.4 Research Contributions . . . 2
- 1.5 Thesis Organization . . . 3

2 Literature Review & Theoretical Background **4**
- 2.1 Literature Review . . . 4
  - 2.1.1 LiDAR Point Clouds and the SemanticKITTI Benchmark . . . 4
  - 2.1.2 Deep Segmentation of LiDAR Point Clouds . . . 6
  - 2.1.3 Classical Geometric Segmentation . . . 7
  - 2.1.4 Object Level Point Cloud Completion . . . 10
  - 2.1.5 Positioning of This Work . . . 14

3 Proposed Methodology **17**
- 3.1 System Architecture . . . 17
- 3.2 Detection Pipeline . . . 18
  - 3.2.1 Overview of the Modular Detection Pipeline . . . 19
  - 3.2.2 Preprocessing: Filtering, Downsampling, and Denoising . . . 19
  - 3.2.3 Ground Plane Removal by RANSAC . . . 20
  - 3.2.4 Object Candidate Clustering with HDBSCAN . . . 20
  - 3.2.5 Geometric Filtering of Non-Car Clusters . . . 21
  - 3.2.6 The Car / Not-Car Point Classifier . . . 22
  - 3.2.7 Multi-Frame Tracking and Track Level Filtering . . . 23
  - 3.2.8 The Frozen Pipeline Configuration . . . 24
- 3.3 Object Completion . . . 25
  - 3.3.1 Completion Targets: Single Frame Car Clusters . . . 26
  - 3.3.2 The Completion Network (PCN) . . . 27
  - 3.3.3 Training Data: KITTI-Like Partials . . . 27
  - 3.3.4 Inference Time Partial Normalisation . . . 29
  - 3.3.5 L-Shape Gating of Completion Inputs . . . 29
  - 3.3.6 Per Car Length Estimation . . . 30
  - 3.3.7 The End-to-End Completion Path . . . 31
  - 3.3.8 Negative Results and Lessons . . . 31
- 3.4 Summary and Methodological Limitations . . . 33

4 Experimental Evaluation **34**
4.1 Dataset & Protocol . . . 34
4.1.1 Baseline Methods . . . 37
4.1.2 Evaluation Metrics . . . 37
4.1.3 Implementation Details . . . 38
4.2 Detection Evaluation . . . 39
4.2.1 Detection and Classification Results . . . 39
4.2.2 Ablation Studies . . . 43
4.2.3 Result Analysis and Error Analysis . . . 45
4.2.4 Runtime Evaluation . . . 51
4.3 Completion Evaluation . . . 52
4.3.1 Completion Results . . . 52
4.3.2 Ablation Studies . . . 56

5 Conclusion and Future Work **57**
5.1 Conclusion . . . 57
5.2 Limitations and Discussion . . . 58
5.2.1 Established Results and Their Reach . . . 58
5.2.2 Validity of the Evaluation . . . 59
5.2.3 Scope of the Completion Claims . . . 59
5.2.4 Ground Truth and Dataset Assumptions . . . 60
5.2.5 Architectural and Operational Limits . . . 60
5.3 Future Work . . . 60

List of Figures

2.1 Properties of a single LiDAR scan 5
2.2 Classical ground removal and clustering 8
2.3 Object level point cloud completion 11
2.4 Point level IoU for detection 13
2.5 Completion fidelity distances 14

3.1 System architecture 18
3.2 Detection pipeline 19
3.3 Clustering stage 21
3.4 Geometric filtering by OBB gates 22
3.5 Dual branch classifier 23
3.6 Tracker and track level filter 24
3.7 Object completion pipeline 26
3.8 Completion network (PCN) 27
3.9 Training input distribution 28
3.10 L-shape gating of completion inputs 30
3.11 Completions on real cars 32

4.1 Bird's-eye view of detections 40
4.2 Per frame detection behaviour 41
4.3 Detection failure close ups 48
4.4 Plausibility by sequence 54

List of Tables

2.1 Real LiDAR completion metrics by type . . . 12
2.2 Positioning against representative SemanticKITTI methods . . . 15

3.1 Frozen pipeline configuration . . . 25

4.1 Dataset roles and modalities . . . 35
4.2 SemanticKITTI sensor specification . . . 35
4.3 Labelled SemanticKITTI sequences and their roles . . . 35
4.4 IoU threshold sensitivity . . . 38
4.5 Unsupervised and supervised stages of the framework . . . 39
4.6 Reported detection operating point . . . 39
4.7 Leave-one-sequence-out cross-validation . . . 41
4.8 Detection stratified by range-to-ego . . . 43
4.9 Stage ablation . . . 43
4.10 Cross-domain classifier transfer . . . 44
4.11 Clustering method benchmark . . . 45
4.12 Ground truth cars across HDBSCAN clusters . . . 46
4.13 Recall repair strategies . . . 49
4.14 Effect of coarsening the clustering voxel . . . 50
4.15 Per frame runtime by stage . . . 52
4.16 Independent classifier plausibility of completed cars . . . 53
4.17 Plausibility acceptance by sequence . . . 54
4.18 Reference free fidelity metrics . . . 55

Chapter 1

Introduction

1.1 Background and Rationale

Autonomous driving depends on a perception system that can recover, several times a second, the three dimensional layout of the scene around the vehicle: where the other road users are, how large they are, and which way they face. LiDAR is central to this task. It measures range directly, returning an unordered cloud of over a hundred thousand points per scan, largely invariant to lighting and carrying the metric depth that motion planning needs. The benchmarks that opened the field were built around it: the KITTI suite [1] paired a scanning LiDAR with calibrated cameras, and its point-wise successor SemanticKITTI [2] turned the same sequences into the dense segmentation benchmark much of the field is now measured against.

The dominant paradigm is to train a deep network end to end on annotated scans: point-based segmentation networks [3, 4, 5], projection- and voxel-based models [6, 7, 8], single pass object detectors [9, 10], and panoptic models [11, 12]. These define the state of the art in accuracy, which is paid for in two costs. The first is supervision: the results rest on dense, point-wise labels, and the annotation behind a benchmark like SemanticKITTI, a class on every one of billions of points, is out of reach for most groups and most new datasets. The second is opacity: an end-to-end network spreads each decision across millions of weights, so when it misses a car there is no single stage to inspect or repair.

This thesis takes the opposite starting point, asking how far a modular pipeline that requires zero additional manual annotation can go: classical geometry for the parts geometry solves well, and small learned models only where discrimination needs them, an approach to LiDAR vehicle detection that has drawn renewed interest [13]. Clustering proposes candidate objects, a small binary car/not-car classifier keeps the vehicles, and a PCN network completes each detected car. Every stage is separately inspectable. The classifier is trained on supervision auto-mined from existing semantic datasets, and the completion network on synthetic shapes alone. Chapter 3 gives the full pipeline (Figure 3.2).

1.2 Completing Occluded Vehicles on Real LiDAR

It is the second half of this pipeline that carries the harder question. Completing a partially observed car, recovering the far side the sensor cannot reach, is attractive for downstream box estimation, and networks such as PCN [14] are well studied on synthetic benchmarks. Carrying them onto real LiDAR raises two coupled difficulties. The first is training without paired supervision: no dataset holds a real occluded car beside its

complete shape, so the network is trained on synthetic CAD partials and must transfer to real scans it never saw during training. The second is evaluation: on real LiDAR there is no clean ground truth complete shape to score against, and the obvious substitute of accumulating the same static car across many frames is itself occluded on the same side, so reference free fidelity metrics reward doing nothing at all. The question this thesis puts to the completion stage is therefore not whether it lowers a distance to a reference that does not exist, but whether it produces shapes that read as genuine cars on real data, judged by evidence that does not depend on a self-made reference.

1.3 Research Objectives

The work is organised around two research questions, one for each half of the framework; two further findings, reported below in their own right, record what the investigation revealed about the detection front end along the way.

1. A Modular Detection Pipeline with Auto-Mined Supervision: can a modular pipeline that leaves geometry to classical methods and learns only where discrimination needs it detect and complete occluded vehicles on real LiDAR, with zero additional manual annotation and every stage separately inspectable? This is answered by the end-to-end detection evaluation, with leave-one-sequence-out cross-validation across all eleven labelled sequences guarding against model selection bias (RQ1; answered in Chapter 4).

2. Car-Plausible Completion on Real LiDAR: does learned completion produce car-plausible shapes on real LiDAR, as judged by an independent shape classifier trained on external complete shape data, and how does it fare on established reference free fidelity metrics? The plausibility judge supplies the favorable real data signal; the fidelity metrics are reported honestly, including where they penalise the completion (RQ2; answered in Section 4.3.1).

1.4 Research Contributions

The thesis makes two contributions, together with the reproducibility infrastructure that supports them.

- A Modular Detection and Completion Framework for Occluded Vehicles: an end-to-end pipeline in which classical geometry does the work geometry solves well and small learned models are used only where discrimination needs them. Ground removal and density clustering propose candidate objects, a binary car/not-car classifier keeps the vehicles, a centroid tracker filters flicker, and a PCN network completes each detected car. Every stage is separately inspectable and its runtime cost is measured rather than estimated. The individual components (RANSAC, HDBSCAN, PointNet, PCN) are established; the contribution is their integration into a coherent, inspectable, reproducible framework for the occluded vehicle problem, evaluated end to end on real LiDAR.

- Auto-Mined Supervision and Synthetic Completion: the framework requires zero additional manual annotation: the classifier's training data is automatically mined

from existing semantic datasets, and the completion network is trained exclusively on synthetic shapes. SemanticKITTI [2] supplies real Velodyne scans for detection, the auto-mined supervision, and evaluation; ShapeNetCore v2 [15] supplies synthetic complete CAD shapes for training the completion network. Because the approach eliminates the need for bounding box annotations, the trained pipeline carries to new sequences without fresh manual labelling. Its real data completion quality is assessed with an independent shape classifier, avoiding any self-made reference.

1.5 Thesis Organization

The remainder of the thesis is organised into five chapters.

Chapter 1: Introduction presents the background and rationale, the challenges of evaluating completion on real LiDAR, the research objectives, the contributions, and the organisation of the thesis.

Chapter 2: Literature Review & Theoretical Background places the pipeline against the deep learning segmentation and point cloud completion literature, carries the positioning table that defends the comparison, and fixes the notation and standard definitions for the operators the pipeline is built from.

Chapter 3: Proposed Methodology describes every stage of the framework: ground removal, clustering, the binary classifier and its training, the centroid tracker, and the point cloud completion network. Each stage is given in its configured, instantiated form, with its input-to-output contract, cross-referencing the general operator definitions in Chapter 2.

Chapter 4: Experimental Evaluation evaluates the framework end to end. It fixes the detection protocol and reports the per stage and cross-validated detection results, the ablations, the distance breakdown, and the runtime. It then root causes the recall shortfall to clustering fragmentation and reports the chain of negative repair strategies, which also bridges to completion. Finally, it evaluates the completion stage through synthetic Chamfer and F-score, the independent classifier plausibility of the completed real clouds, and reference free fidelity reported with its caveat.

Chapter 5: Conclusion and Future Work answers the two research questions, names every weakness the work is subject to, and lays out what remains.

Chapter 2

Literature Review & Theoretical Background

This chapter places the thesis in its research context. It reviews the two bodies of work the pipeline draws on, LiDAR point cloud segmentation and object level point cloud completion, and then positions the specific design choices of this work against them. The organising contrast runs throughout: the dominant approach to LiDAR segmentation is a single end-to-end network trained on dense per point annotation, whereas this thesis builds an interpretable pipeline from training free geometric clustering plus a lightweight learned classifier, and treats completion as a separate, downstream problem. Section 2.1.5 makes that contrast quantitative in a positioning table. Section 2.1 reviews what exists in LiDAR segmentation and object completion and where this work sits within it, and fixes the notation and standard definitions for the operators the pipeline is built from alongside the methods that use them. The pipeline specific architectures, configurations, and design choices of each component are deferred to the method chapter (Section 3 for detection, Section 3.3 for completion).

Scope of the review. The field around automotive LiDAR perception is far larger than any one chapter can survey, so the review is bounded by relevance to the thesis's own design and evaluation. Included are the deep and classical LiDAR segmentation paradigms the detection front end is built from and positioned against (Sections 2.1.2 and 2.1.3), and object level point cloud completion together with its evaluation metrics, the setting of the thesis's contribution (Section 2.1.4). Left deliberately outside are camera–LiDAR fusion and SLAM or map optimisation, both orthogonal to a single scan, LiDAR only object pipeline. Also excluded are full 3D object detection benchmarks scored by bounding box average precision, which measure a different output than the point level clusters this pipeline emits. This exclusion is argued in full where it naturally arises (Section 2.1.5). The conceptual grounding and notation needed to read this related work, but no more, are introduced alongside the methods that use them.

2.1 Literature Review

2.1.1 LiDAR Point Clouds and the SemanticKITTI Benchmark

A rotating LiDAR sensor returns, for each scan, an unordered set of 3D points sampled from the surfaces around the vehicle. The points are sparse, irregularly spaced, and strongly range dependent: nearby objects are dense, distant ones may be a handful of

returns, and every object is seen from a single viewpoint and is therefore partial. These properties (irregularity, sparsity, single view partiality) shape every method discussed below and every difficulty this thesis reports (Figure 2.1).

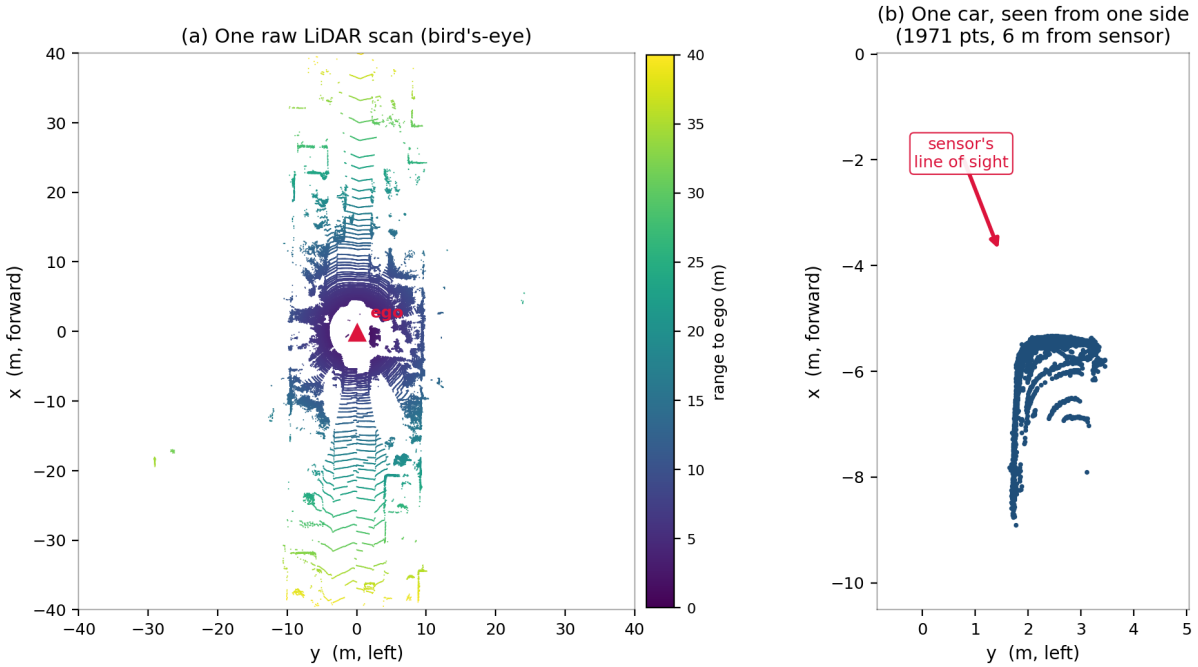


Figure 2.1: A single SemanticKITTI scan. **(a)** Bird's-eye view of one raw scan coloured by range to the ego sensor (red triangle); the central disc is the sensor blind spot. **(b)** A single parked car at 6 m, whose returns form a one-sided "L" leaving the occluded side empty.

The KITTI Vision Benchmark [1] established the standard automotive LiDAR setting (a Velodyne HDL-64E on a car, with synchronised camera and pose). Its segmentation oriented successor, SemanticKITTI [2], provides dense per point semantic labels for the 22 sequences of the KITTI odometry split (sequences 00–10 are released with labels; the labels for sequences 11–21 are withheld and scored only through the online test server). A later extension adds temporally consistent instance identities for a LiDAR-based panoptic benchmark [16]. Three tasks are defined on it. Semantic segmentation assigns every point a class and is scored by the mean intersection over union across classes (mIoU). Instance segmentation separates individual objects within the "thing" classes (car, pedestrian, cyclist, and so on). Panoptic segmentation unifies the two and is scored by panoptic quality (PQ), which factorises into a segmentation quality term (SQ, the mean IoU of matched segments) and a recognition quality term (RQ, an F_1-like matching score). PQ^{Th} restricts PQ to the "thing" classes, and $PQ^{\dagger}$ is a variant that scores "stuff" by IoU. These definitions matter for Section 2.1.5, because they are the metrics the literature reports and none of them is the point level per class F_1 this thesis uses.

The decisive property of SemanticKITTI for this work is its annotation cost. The labels are dense and per point across billions of points, and the panoptic extension adds a manual instance association pass over sequences. This is the resource the mainstream methods below consume and the one this thesis is built to avoid: its segmentation stage requires no labelled point clouds at all, and its classifier is trained on clusters mined and weakly labelled automatically from the pipeline's own output (Chapter 3).

2.1.2 Deep Segmentation of LiDAR Point Clouds

The current benchmark leaders are deep networks trained end-to-end on the dense labels. They differ mainly in how they impose structure on the irregular point set. Projection-based methods rasterise the scan into a 2D range image and apply image convolutions: SqueezeSeg [6] and RangeNet++ [7] are representative, and are fast because they reuse mature 2D machinery, at the cost of discretisation artefacts at depth discontinuities. Point-based methods operate on the raw points through permutation invariant operators: PointNet [3] introduced the shared MLP plus symmetric pooling design, PointNet++ [4] added hierarchical local grouping, and RandLA-Net [5] made the point-based route scale to full driving scans through efficient random sampling. Voxel-based methods discretise 3D space and run sparse convolutions; Cylinder3D [8] uses a cylindrical partition matched to the sensor's polar geometry and reports 67.8 mIoU on the SemanticKITTI test set, among the higher single scan results.

Panoptic methods extend semantic backbones with instance reasoning. Panoptic-PolarNet [11] predicts instances proposal free in a polar bird's-eye-view; DS-Net [12] clusters in a learned embedding via a dynamic shifting module; Panoster [17] learns instance ids end-to-end; and EfficientLPS [18] targets an efficient architecture for the joint task. The dataset authors' own baselines pair a semantic network with a 3D detector to supply instances [16].

Two properties are common to this whole family and frame the thesis's alternative. First, all of them require the dense per point (and, for panoptic, per instance) supervision described above; their accuracy is inseparable from that annotation budget. Second, they are end-to-end networks whose decisions are not individually inspectable , a misclassified region has no stage level explanation. PointNet and PointNet++ are relevant to this thesis for a narrower reason as well: they are the backbone family from which its car/not-car classifier is built (Chapter 3). The classifier, however, is a small binary network applied to already segmented clusters, not a full scene segmentation model, and it is trained without any hand drawn labels.

Permutation Invariant Point Classification

The car/not-car classifier is built on PointNet [3], whose defining property is invariance to the ordering of its input points. A function of a point set is made permutation invariant by applying a shared map to each point, pooling with a symmetric operator, and passing the pooled feature through a final map,

$$f(\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}) = g\Big(\max_{i=1,\ldots,n} h(\mathbf{x}_i)\Big), \tag{2.1}$$

where

$\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$	the input point set,
h	shared per point map (an MLP),
g	map applied to the pooled feature (an MLP),
max	coordinate-wise maximum (symmetric pooling).

Because the max pool is symmetric, f is independent of the order of the points, and h, g are learned end-to-end. Chapter 3 applies this design to a single cluster.

2.1.3 Classical Geometric Segmentation

Before end-to-end learning dominated the benchmarks, LiDAR objects were extracted by geometric reasoning, and that route is the one this thesis takes for the segmentation stage. The standard recipe removes the ground plane, commonly by RANSAC plane fitting, and clusters the remaining points into objects. The clustering step is where the design choice lives. Fixed radius methods such as Euclidean clustering and DBSCAN [19] group points within a single distance threshold ε; this is fast and fully interpretable, but a single ε cannot serve both the dense near field and the sparse far field of a LiDAR scan. HDBSCAN [20], in the widely used implementation of [21], replaces the fixed radius with a hierarchy over density levels and extracts clusters that are stable across scales, which is better matched to range varying point density.

This clustering plus classification recipe remains an active design point wherever interpretability and real-time efficiency are valued over benchmark accuracy. Recent work for autonomous racing, for instance, pairs bird's-eye-view clustering with a lightweight machine learning vehicle classifier to reach long range detection without a deep dense feature network [13]. The pipeline in this thesis is of the same family, ground removal, clustering, then a learned per object classifier, applied to the SemanticKITTI setting (Figure 2.2).

The appeal of this route is that it needs no training data for segmentation and every stage is inspectable. Its weaknesses are equally concrete and are documented directly in this thesis: density-based clustering has no notion of objectness, so it fragments a single car into several clusters when internal density gaps appear, and this fragmentation, not classification error, is the dominant recall limiter (Section 4.2.3). The positioning below should be read with that trade in view: the interpretability and minimal supervision of the classical route are bought at a measurable recall cost relative to the supervised networks.

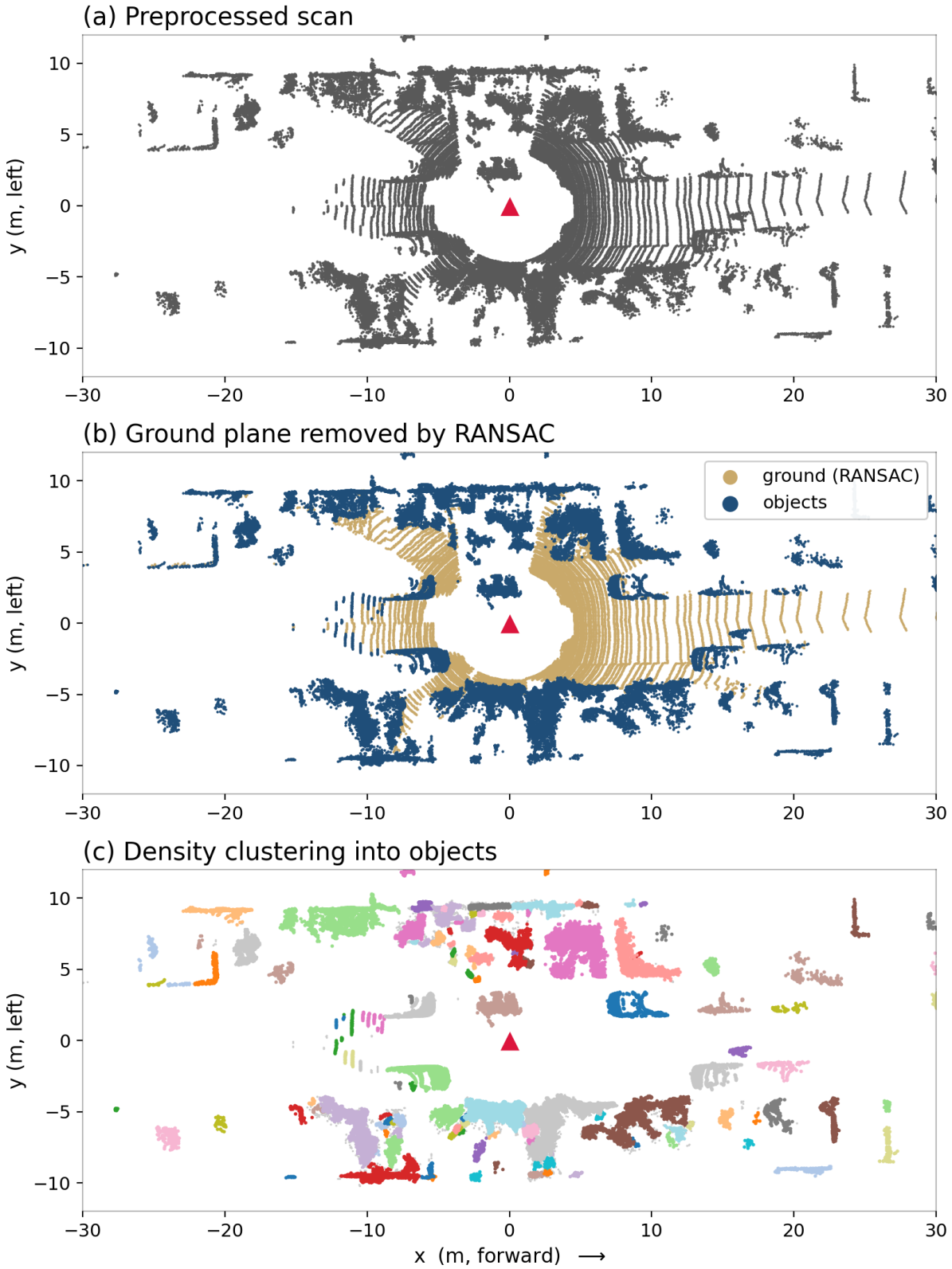


Figure 2.2: The classical geometric recipe this thesis adopts, on one SemanticKITTI scan (bird's-eye). **(a)** The preprocessed scan. **(b)** RANSAC fits and removes the ground plane (tan), leaving object points (blue). **(c)** Density-based clustering groups the object points into candidate objects; colours are cluster identities.

A single LiDAR scan is written as a finite point set $P = \{\mathbf{x}_i \in \mathbb{R}^3\}_{i=1}^{N}$.

Ground Plane Removal by RANSAC

A plane is a pair $(\mathbf{n}, d)$, and a point's signed distance to it is

$$\delta(\mathbf{x}; \mathbf{n}, d) = \mathbf{n} \cdot \mathbf{x} + d, \tag{2.2}$$

where

$\mathbf{n}$	plane unit normal, $\|\mathbf{n}\| = 1$,
d	plane offset,
$\mathbf{x}$	a query point, $\mathbf{x} \in \mathbb{R}^3$.

RANSAC [22] fits the ground plane by repeatedly sampling three points, forming the plane through them, and scoring it by its inlier set,

$$\mathcal{I}(\mathbf{n}, d) = \Big\{ \mathbf{x} \in P : |\delta(\mathbf{x}; \mathbf{n}, d)| \le \tau_g \Big\}, \tag{2.3}$$

where

$\mathcal{I}(\mathbf{n}, d)$	inlier set of the plane $(\mathbf{n}, d)$,
τ_g	inlier distance tolerance,
P	the input scan.

The candidate with the largest inlier set is kept and its inliers removed as ground. The number of iterations needed to draw at least one all inlier sample is

$$T = \left\lceil \frac{\log(1-p)}{\log\big(1 - w^s\big)} \right\rceil, \tag{2.4}$$

where

T	number of RANSAC iterations,
p	target probability of a clean sample,
w	inlier fraction of the scan,
s	sample size, $s = 3$.

Voxel Grid Downsampling

To reduce the range dependent density bias before clustering, the non-ground points are downsampled on a regular grid. Each point maps to an integer cell index

$$\mathrm{vox}(\mathbf{x}) = \left\lfloor \frac{\mathbf{x} - \mathbf{x}_{\min}}{v} \right\rfloor \in \mathbb{Z}^3, \tag{2.5}$$

the floor taken per coordinate, where

v	voxel side length,
$\mathbf{x}_{\min}$	per axis minimum corner of the point set.

Writing $P_c = \{\mathbf{x} \in P : \mathrm{vox}(\mathbf{x}) = c\}$ for the points falling in cell c, every occupied cell is replaced by their centroid,

$$\bar{\mathbf{x}}_c = \frac{1}{|P_c|} \sum_{\mathbf{x} \in P_c} \mathbf{x}, \tag{2.6}$$

where

$\bar{\mathbf{x}}_c$	centroid representing cell c,
P_c	the points falling in cell c.

Density-Based Clustering: DBSCAN and HDBSCAN

The non-ground points are grouped into objects by density. DBSCAN [19] uses a fixed radius ε and a minimum count m: a point is a core point when its neighbourhood is large enough,

$$|\mathcal{N}_\varepsilon(\mathbf{p})| \geq m, \tag{2.7}$$

where

$\mathcal{N}_\varepsilon(\mathbf{p})$	ε-neighbourhood $\{\mathbf{q} \in P : \|\mathbf{p} - \mathbf{q}\| \leq \varepsilon\}$,
ε	neighbourhood radius,
m	minimum neighbour count.

Clusters are the maximal sets connected through core points, and a single ε cannot serve both the dense near field and the sparse far field of a scan. HDBSCAN [20, 21] removes the fixed radius, replacing Euclidean distance with the mutual reachability distance

$$d_{\text{mreach}}(\mathbf{a}, \mathbf{b}) = \max\Big(d_{\text{core}}(\mathbf{a}),\ d_{\text{core}}(\mathbf{b}),\ \|\mathbf{a} - \mathbf{b}\| \Big), \tag{2.8}$$

where

$d_{\text{mreach}}(\mathbf{a}, \mathbf{b})$	mutual reachability distance of $\mathbf{a}, \mathbf{b}$,
$d_{\text{core}}(\mathbf{p})$	distance from $\mathbf{p}$ to its m-th nearest neighbour.

It builds a minimum spanning tree under d_{mreach} and condenses it into a hierarchy indexed by $\lambda = 1/d_{\text{mreach}}$. A candidate cluster's stability is

$$S(C) = \sum_{\mathbf{p} \in C} \Big(\lambda_{\mathbf{p}} - \lambda_{\min}(C) \Big), \tag{2.9}$$

where

$S(C)$	stability of candidate cluster C,
$\lambda_{\min}(C)$	density level at which C is born,
$\lambda_{\mathbf{p}}$	level at which point $\mathbf{p}$ leaves C.

The flat clustering is the set of disjoint clusters of maximum total stability. Stability-based extraction is what lets one setting cluster both near and far cars; it is also, as Section 4.2.3 shows, what splits a single car when an internal density gap makes two sub-clusters individually more stable than the whole.

2.1.4 Object Level Point Cloud Completion

The second phase of the pipeline reconstructs a denser shape from the sparse, single view partial cloud of a detected car. Point cloud completion learns this partial-to-complete mapping. PCN [14] established the coarse-to-fine design, a PointNet encoder producing a global shape code, decoded first to a coarse set and then folded to a dense output, and remains a strong, compact baseline. Later work improves detail through attention and progressive refinement: PoinTr [23] casts completion as set-to-set translation with a geometry aware transformer, SnowflakeNet [24] generates points by hierarchical "snowflake" deconvolution, and SeedFormer [25] introduces patch seed representations. These models are trained on synthetic CAD shapes, most commonly ShapeNet [15], with partial inputs produced by virtual rendering. Concretely (Figure 2.3), on a real single frame car the completion maps the one-sided partial the sensor observes into a full car shape: this is the partial-to-complete mapping the thesis's completion stage performs (Section 3.3), and how well it does so on real data is evaluated in Section 4.3.1.

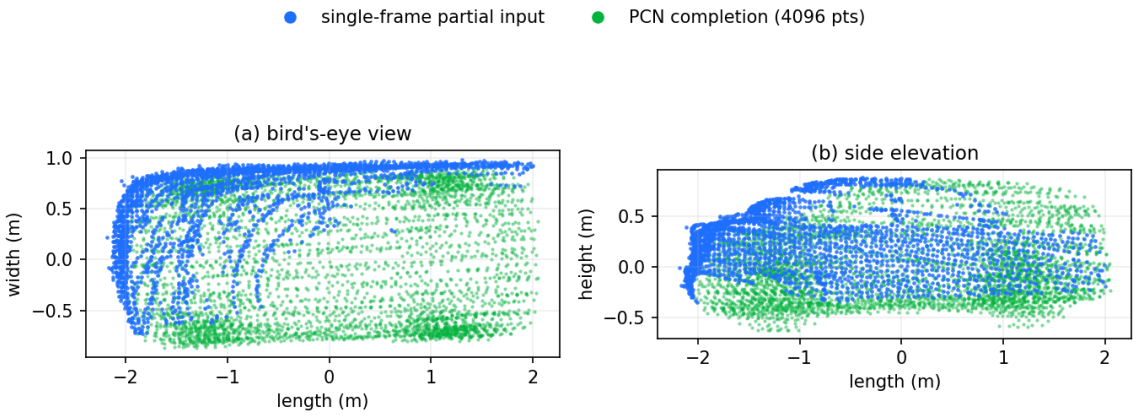


Figure 2.3: Object-level point cloud completion on a real LiDAR car, in bird's-eye view (a) and side elevation (b). The single-frame partial (blue) covers only the near faces the sensor observes; the PCN completion (green) fills the full car surface, including the occluded side.

Two points from this literature bear directly on later chapters. First, models trained on clean synthetic partials face a synthetic-to-real domain gap when applied to noisy, sparse LiDAR clusters. This thesis adopts PCN and confronts that gap head on (Section 3.3), and its comparison of PCN against a transformer alternative found no decisive real data advantage for the heavier model (Section 4.3.1). Second, and more consequentially, the standard completion metric, Chamfer distance to a reference shape, presupposes a complete ground truth surface, which real accumulated LiDAR does not provide: the accumulation is itself one-sided, seen from the same road, so scoring a completion against it rewards a network that reproduces the observed side and adds nothing. This is why real LiDAR completion is not evaluated by reference Chamfer at all in the recent literature, and why this thesis evaluates it without any self-made reference (Section 4.3.1). This should be distinguished from semantic scene completion, which fills a whole voxelised scene from a single scan; the present work completes individual segmented objects, a different and narrower task.

Because real automotive LiDAR offers no complete target, a small toolkit of completion metrics has grown up around the KITTI cars, of two kinds. The first kind is reference free fidelity: the unidirectional Chamfer and Hausdorff distances (UCD and UHD), which score a completion by how closely it reproduces the points of its own partial input. Chen et al. [26] apply UHD directly to KITTI cars, and P2C [27] uses UCD together with a region aware variant (RCD) that localises the comparison around observed skeleton points but still anchors it to seen geometry. PCN [14] scores its KITTI completions by Fidelity (how well the input is preserved in the output), Minimal Matching Distance (Chamfer distance to the nearest complete car in a synthetic reference set), and Consistency (agreement between the completions of one instance across consecutive frames). Ren et al. [28] score completed vehicles against a multi-frame accumulation with Chamfer-based distances. Every one of these references something other than the real occluded surface: the observed input, a synthetic prior, the model's own outputs, or a one-sided accumulation. Crucially, a completion that merely copies its input scores best on the input referenced measures, so none of them rewards recovering genuinely unobserved surface. The second kind exists precisely to close that gap: a plausibility term that asks not how close a completion is to a reference but whether it reads as a real object at all. Chen et al. [26] pair UHD with exactly such a term, the acceptance rate of a shape classifier trained on external complete

shapes (ModelNet40), chosen to avoid bias toward the ShapeNet shapes the completion network was trained on. This thesis adopts that plausibility protocol as its favorable real data completion signal, and reports fidelity alongside it with the caveat that fidelity rewards under-completion (Section 4.3.1).

Table 2.1 summarises this landscape against the two properties the completion claim needs: whether a metric credits a completion for producing a plausible whole object rather than reproducing its input, and what each scores a completion against. The fidelity measures reward input reproduction; the plausibility term is the one that credits a car-plausible completed shape.

Table 2.1: Established real LiDAR completion metrics, by kind and by what each scores a completion against. The final row is the plausibility term this thesis adopts (Section 4.3.1).

Metric	Kind	Scored against	Plausible shape
UCD [27]	fidelity	observed input	no
UHD [26]	fidelity	observed input	no
Fidelity [14]	fidelity	observed input	no
MMD [14]	reference	synthetic CAD prior	indirect
Consistency [14]	self-ref.	model's own frames (cross-frame)	no
meanCD [28]	reference	full multi-frame accumulation	no
Plausibility [26]	plausibility	external classifier's car acceptance	yes

Point Set Distances and Completion Quality Measures

Completion and its evaluation are expressed through distances between point sets. The Chamfer distance between sets A and B averages, in each direction, the squared distance from every point to the nearest point of the other set,

$$\mathrm{CD}(A,B) = \frac{1}{|A|}\sum_{\mathbf{a}\in A}\min_{\mathbf{b}\in B}\|\mathbf{a}-\mathbf{b}\|^2 \;+\; \frac{1}{|B|}\sum_{\mathbf{b}\in B}\min_{\mathbf{a}\in A}\|\mathbf{b}-\mathbf{a}\|^2, \tag{2.10}$$

where

A, B	the two point sets compared,
$\|\mathbf{a}-\mathbf{b}\|$	Euclidean distance between two points.

It is the standard training objective and synthetic benchmark score for completion, where a complete ground truth shape B exists [29, 14]. On real LiDAR no such B exists (Section 2.1.4), so completion is scored by two reference free measures instead. Detection is first scored by a point level intersection over union between a predicted cluster Q and a ground truth car's point set G,

$$\mathrm{IoU}(Q,G) = \frac{|Q\cap G|}{|Q\cup G|}, \tag{2.11}$$

where

Q	predicted cluster (its point set),
G	ground truth car's point set,
$\|\cdot\|$	number of points in a set.

A prediction matches a ground truth object when $\mathrm{IoU}(Q, G) \geq 0.3$ (Chapter 4; Figure 2.4).

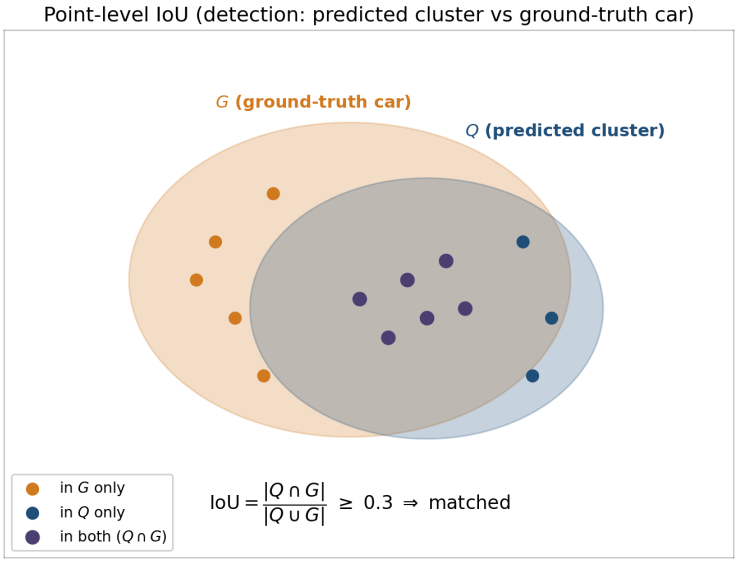


Figure 2.4: Point level IoU, the detection matching metric: the overlap between a predicted cluster Q and a ground truth car's point set G, as the fraction of their union the two share; a prediction is matched when $\mathrm{IoU} \geq 0.3$.

For completion, unidirectional fidelity measures how well a completed cloud C preserves the observed partial input P, by the directed distance from each observed point to its nearest completed point. The unidirectional Chamfer distance (UCD) [27] averages it and the unidirectional Hausdorff distance (UHD) [26] takes its worst case,

$$\mathrm{UCD}(P, C) = \frac{1}{|P|} \sum_{\mathbf{p} \in P} \min_{\mathbf{c} \in C} \|\mathbf{p} - \mathbf{c}\|^2, \qquad \mathrm{UHD}(P, C) = \max_{\mathbf{p} \in P} \min_{\mathbf{c} \in C} \|\mathbf{p} - \mathbf{c}\|. \tag{2.12}$$

where

P	observed partial input,
C	completed cloud,
$\min_{\mathbf{c} \in C} \|\mathbf{p} - \mathbf{c}\|$	distance from $\mathbf{p}$ to its nearest completed point.

Both are smallest when C reproduces P exactly, so they reward preserving the observed side and cannot, alone, credit added surface, the caveat under which Section 4.3.1 reports them (Figure 2.5).

The second measure is plausibility [26]: an independent shape classifier $\phi(\cdot)$, trained on external complete shapes and returning a probability per class, judges whether a completed cloud reads as a car. Over a set of completions $\{C_k\}_{k=1}^{K}$, the acceptance rate is the fraction the classifier assigns to the car class,

$$\mathrm{Acc} = \frac{1}{K} \sum_{k=1}^{K} \mathbf{1}\left[\arg\max_{j} \phi_j(C_k) = \mathrm{car} \right], \tag{2.13}$$

where

$\phi_j(C_k)$	classifier probability of class j for completion C_k,
K	number of completions scored,
$\mathbf{1}[\cdot]$	indicator, 1 when the top class is car.

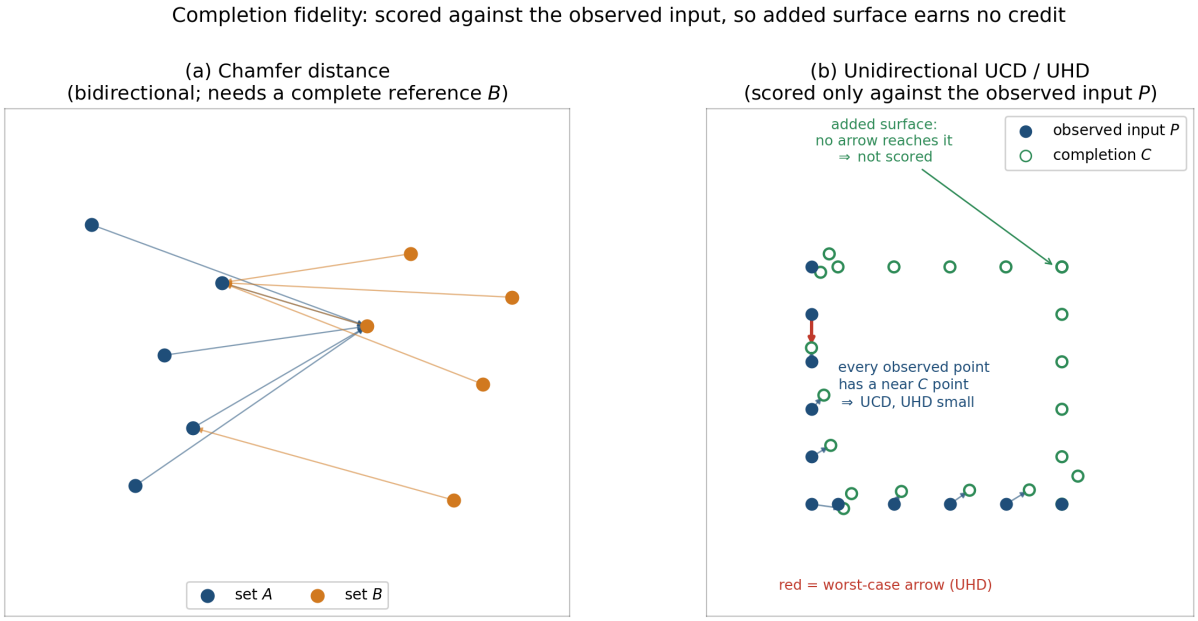


Figure 2.5: The two completion fidelity distances, on toy sets. **(a)** Chamfer distance sums nearest-point distances in both directions, so it needs a complete reference B. **(b)** The unidirectional UCD and UHD score a completion C only against the observed input P, so the added surface on the unobserved side receives no arrow and earns no credit.

It is reported together with the mean car probability $\frac{1}{K}\sum_k \phi_{\text{car}}(C_k)$. Because ϕ is trained on shapes disjoint from the completion network's training set, a high acceptance rate is evidence that completion produces genuinely car-plausible shapes rather than shapes that merely resemble its own training data (Section 4.3.1).

2.1.5 Positioning of This Work

The pipeline studied here is a car only instance detector built from training free clustering and a binary classifier, followed by object completion. It occupies a different point in the design space from the benchmark leaders of Section 2.1.2. Table 2.2 sets that position out along the axes on which it actually differs, approach, segmentation supervision, and output type , rather than on an accuracy score. A shared accuracy column is deliberately omitted: the published methods are scored by mIoU and panoptic quality across all classes, this work by a car only point level F_1, and placing those figures in one column would invite a comparison the metrics do not support. The accuracy gap is real, and is stated plainly in the text below the table instead.

Table 2.2: Positioning against representative SemanticKITTI methods by approach, segmentation supervision, and output type. A shared accuracy column is omitted by design; the accuracy gap is stated in the text.

Method	Approach	Supervision	Output
RangeNet++ [7]	Range image CNN, semantic	Dense per point	Per point class
RandLA-Net [5]	Point-based, semantic	Dense per point	Per point class
Cylinder3D [8]	Voxel (cylindrical), semantic	Dense per point	Per point class
RangeNet++ & PointPillars [16]	Semantic net + detector, panoptic	Dense per point + boxes	Per point class + instance
Panoptic-PolarNet [11]	End-to-end panoptic	Dense per point + instance	Per point class + instance
DS-Net [12]	End-to-end panoptic	Dense per point + instance	Per point class + instance
EfficientLPS [18]	End-to-end panoptic	Dense per point + instance	Per point class + instance
This work	Geometric clustering + binary classifier (+ completion)	None for segmentation; auto-mined weak labels for the classifier	Point level car clusters (+ completion)

Read as requirements, the table states the thesis's position plainly. On their own benchmarks the supervised networks reach 52.2 to 67.8 mIoU (semantic) and 37.1 to 57.4 PQ (panoptic) over the official test sequences 11–21, scored across all classes. This work reports a car only point level F_1 of 0.812 (recall 0.732 against the $\geq$-10-surviving points denominator of Chapter 4, precision 0.911) on sequence 08. These are not the same measurement: none of the published metrics is this work's point level car F_1, and the published rows are all class test set scores while this work's is car only and single sequence. The lower accuracy is therefore a difference of paradigm, not a lower score on a shared axis, and this work does not contest the gap. What the pipeline offers instead is a different set of properties. It segments without any labelled point clouds, runs on a single consumer GPU without a training stage for detection (its runtime is reported in Section 4.2.4), and keeps every stage of its decision inspectable. Its failure modes can therefore be root caused rather than left opaque: the recall shortfall is traced to clustering fragmentation, not classifier error, in Section 4.2.3. The accuracy gap is therefore a deliberate consequence of choosing interpretability and minimal supervision over benchmark accuracy, not an unexplained deficiency.

A related question the table anticipates is why the comparison is drawn against segmentation methods rather than 3D object detection, the task the "detector" label most naturally invokes. Detection benchmarks score oriented 3D bounding boxes by average precision, as in PointPillars [9] and CenterPoint [10]. This pipeline, however, emits point level clusters , it must, because those clusters are the input to the completion stage (Section 3.3), and never fits an oriented box; box AP would require a box fitting stage the pipeline deliberately does not have. Point level IoU against the ground truth car points is

therefore the metric that matches the actual output, and the choice of IoU threshold and recall denominator that make that measurement precise, together with its sensitivity to those choices, is examined directly in Section 4.1.

This framing also sets the boundary of the thesis's contribution. The individual building blocks are all established, and are reimplemented and tuned here rather than invented: classical ground removal plus clustering detection, including recent clustering plus classifier detectors for LiDAR vehicles [13]; a lightweight multi-object tracking stage used only to filter transient detections across frames (in the tradition of 3D multi-object trackers such as AB3DMOT [30], and not a contribution of this work); and PCN-style completion. The contribution is their integration: a modular, inspectable framework that detects occluded vehicles and completes them on real LiDAR, evaluated end to end, with its completion quality assessed by an independent classifier so that no self-made reference is involved (Section 4.3.1). Two diagnostic findings sit alongside it: the root cause analysis of the recall bottleneck together with the chain of negative repair results (Section 4.2.3), and the finding that synthetic pretraining is redundant for the classifier at scale (Chapter 4). The positioning table is the backdrop against which the contribution is argued: this is an interpretable system built on auto-mined supervision, whose value is in what it explains and measures, not in topping a benchmark it was never designed to win.

Three points emerge from this review and set up the rest of the thesis. First, the accuracy of mainstream LiDAR segmentation is inseparable from a dense per point annotation budget, which motivates the interpretable clustering plus classifier route taken here, built on auto-mined supervision, and whose recall cost the later chapters measure and root cause. Second, that route's stage level transparency is what makes such root causing possible at all, where an end-to-end network offers no stage to inspect. Third, completion on real LiDAR has no valid reference-based score: every reference or fidelity metric is anchored to the observed input, a synthetic prior, or a one-sided accumulation, and rewards a completion that reproduces its input, so the field pairs them with a plausibility term that asks whether the completed shape reads as a real object. These three points are the basis on which the thesis is built: the interpretable framework the first two motivate, and the plausibility-based completion evaluation the third calls for, are what its two research questions address (Chapter 1). The chapter has also fixed the notation and standard operators the later chapters build on. Chapter 3 turns those operators into the detection pipeline and the completion method, in their configured, instantiated form.

Chapter 3

Proposed Methodology

This chapter presents the proposed method: a modular detection and completion framework for automotive LiDAR. Section 3.1 gives the overall architecture; the sections that follow describe each stage in the order a scan flows through it, in its configured, instantiated form. The general operators each stage is built from are defined in Chapter 2; this chapter states how each is set up and connected here, and the end-to-end evaluation is Chapter 4.

3.1 System Architecture

The framework is a single system that turns a raw LiDAR scan into completed, box estimated cars. It is organised as two stages in sequence, each the subject of one section below. The detection stage (Section 3.2) segments and classifies car clusters from the raw scan, emitting an oriented bounding box and a stable track identity per car. The completion stage (Section 3.3) reconstructs each detected car's full surface from the one-sided partial a single frame observes. Figure 3.1 shows this two-stage organisation, and Figure 3.2 details the detection chain that produces every input the completion stage consumes.

The design is deliberately modular and classical: classical geometry does the work geometry solves well, and small learned models are used only where discrimination needs them, so every stage is separately inspectable and its cost separately measurable. This architecture is what lets the evaluation (Chapter 4) attribute the recall limit to a single stage rather than to an opaque network.

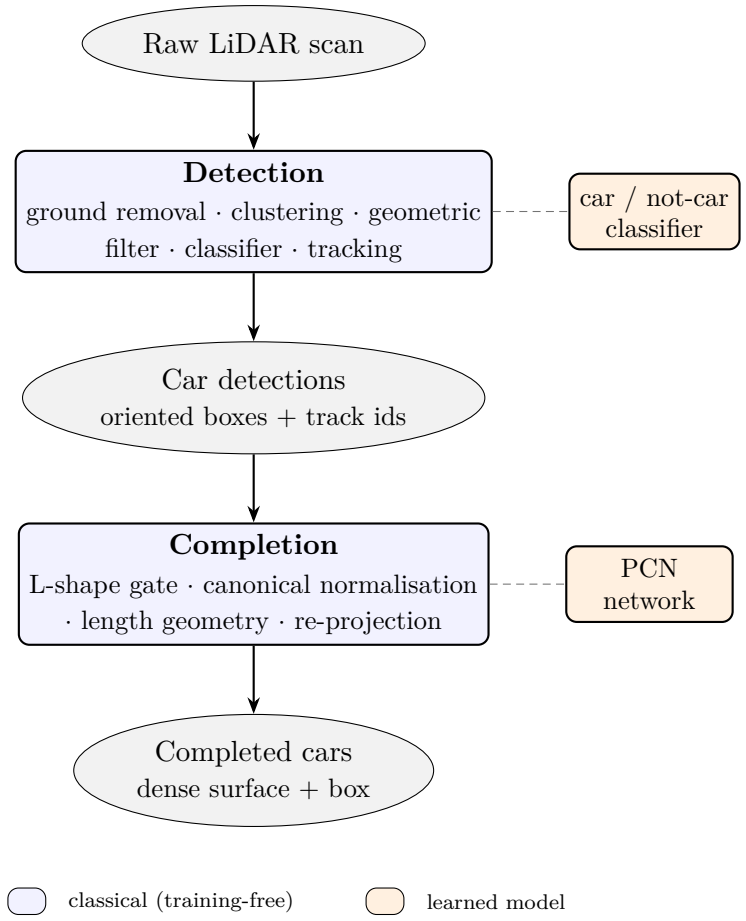


Figure 3.1: The two-stage system architecture: a detection stage (Section 3.2) feeds a completion stage (Section 3.3). Both are built mostly from classical geometry with a single small learned model each (shaded): the car/not-car classifier and the PCN network. Figures 3.2 and 3.7 expand the two stages.

3.2 Detection Pipeline

This section describes the car detection pipeline that produces every detection evaluated in Chapter 4 and every cluster completed in Section 3.3. The design is deliberately modular and classical: a chain of independent stages, each with a small number of interpretable parameters, rather than an end-to-end learned detector. This choice trades peak accuracy for interpretability, reproducibility, and zero additional manual annotation: the classifier is trained on labels mined automatically from existing semantic annotations (Section 3.2.6). It is also what lets Section 4.2.3 root cause the recall shortfall to a single stage rather than to an opaque network.

The pipeline takes one raw LiDAR scan and emits a set of car detections, each an oriented bounding box with a stable track identity across frames. Every stage parameter lives in a single frozen configuration, fixed before the write up began; the frozen values are collected in Table 3.1. Figure 3.2 gives the full chain.

3.2.1 Overview of the Modular Detection Pipeline

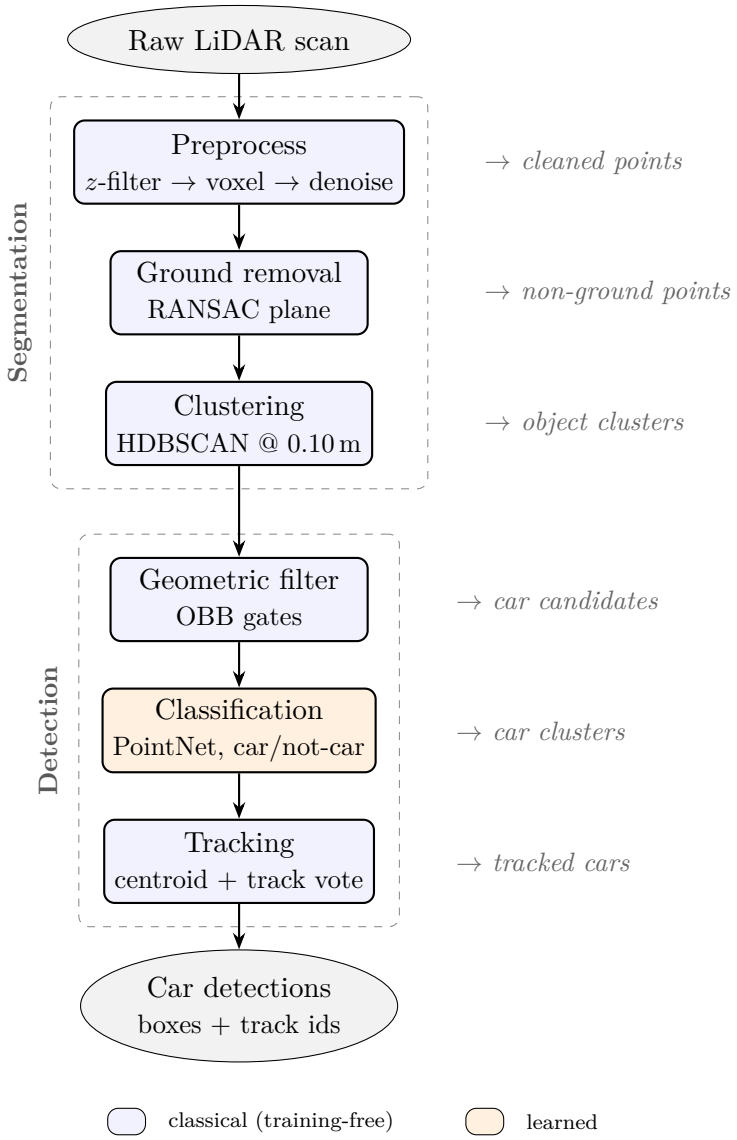


Figure 3.2: The detection pipeline, grouped into two phases (dashed): a segmentation phase that turns a raw scan into clusters (preprocess, ground removal, clustering) and a detection phase that turns clusters into tracked car boxes (geometric filter, classification, tracking), with each stage's output shown at right. Learned stages are shaded; all others are classical.

The system is offline by design: the tracker and the track level filter use the whole sequence, and real-time operation is out of scope. The pipeline runs well below sensor rate, and the per stage runtime budget, which stages dominate, and where the learned and classical costs fall, is measured and reported in the runtime section of Chapter 4 rather than claimed here.

3.2.2 Preprocessing: Filtering, Downsampling, and Denoising

Input: raw LiDAR scan | **Output:** cleaned point cloud (height-filtered, voxel-downsampled, denoised)

Each raw scan passes through three cleaning steps before any object reasoning:

1. Ground height z-filter. Points below $z = -2.0\,\text{m}$ in the sensor frame are dropped. This removes the bulk of the road surface and any sub-ground noise cheaply, before the more expensive steps see them.

2. Voxel downsampling at 0.05 m. Points are collapsed to one representative per 5 cm voxel, which both regularises the range dependent density of LiDAR and cuts the point count by roughly an order of magnitude.

3. Statistical outlier removal. Points whose mean distance to their 20 nearest neighbours exceeds 2.0 standard deviations of the local distribution are discarded, removing isolated speckle.

The order of the last two steps is behaviour changing. In the frozen configuration (`voxel_before_denoise = True`) downsampling runs before denoising, so the statistical outlier test operates on roughly ten times fewer points, and on a regularised point set, which changes which points survive. This ordering was adopted as part of the pipeline optimisation primarily because it improves detection (Chapter 4). It is behaviour changing, so we flag it here rather than presenting the two orderings as equivalent.

3.2.3 Ground Plane Removal by RANSAC

Input: cleaned point cloud | **Output:** non-ground (object) points + fitted ground plane

The dominant remaining structure after preprocessing is the ground plane, which must be removed before clustering or it bridges every object into one component. We fit a single plane with RANSAC: over 300 iterations, three points are sampled, their plane computed, and inliers counted within 0.2 m; the best supported plane is kept. The plane is accepted only if its normal is near vertical, the z-component of the unit normal must exceed 0.5, so a spuriously fitted vertical surface (a wall, the side of a truck) cannot be mistaken for ground. If the check fails, no points are removed and the whole cloud is passed on as objects; this fail safe is rare but prevents a bad plane fit from deleting real cars.

A single global plane is a deliberate simplification: it assumes the local ground is approximately planar, which holds on the flat urban sequences used here but would degrade on strong slopes. A grid-based ground model (Patchwork++-style) is listed as future work, not a limitation we claim to have solved. The iteration count was reduced from 1000 to 300 as part of the runtime optimisation after confirming the plane fit converges well before 1000 iterations on this data. The fit uses a fixed seed (`np.random.default_rng(42)`), so ground removal, and therefore the whole pipeline, is deterministic across runs.

3.2.4 Object Candidate Clustering with HDBSCAN

Input: non-ground points | **Output:** object clusters

Non-ground points are grouped into object candidates with HDBSCAN (Section 2.1.3 gives the general mutual reachability construction), here configured with a minimum cluster size of 10 and a minimum samples value of 5 (the neighbour count m that sets the core distance). HDBSCAN is chosen over a fixed radius method because LiDAR density falls off with range: a single Euclidean radius that merges nothing at 5 m will

over-segment at 40 m. HDBSCAN's variable density hierarchy adapts to this without a hand tuned radius schedule, and Chapter 4 reports a controlled benchmark confirming it beats DBSCAN and Euclidean extraction on recall at matched settings.

Two details of the frozen configuration are load bearing. First, clustering runs on a coarser 0.10 m voxel grid than the 0.05 m preprocessing grid: the object cloud is re-downsampled to 0.10 m, HDBSCAN is run there, and the labels are propagated back to the full resolution points by nearest neighbour. This was adopted because the coarser grid closes intra-car density gaps that otherwise cause a single car to split into several clusters, the central mechanism of the recall bottleneck analysed in Section 4.2.3 (Figure 3.3); its measured effect on true positives is reported there. Second, HDBSCAN is by far the pipeline's most expensive stage, because it is superlinear in the point count, so the coarse grid re-voxelisation is as much a speed measure as an accuracy one; the per stage runtime is reported in Chapter 4.

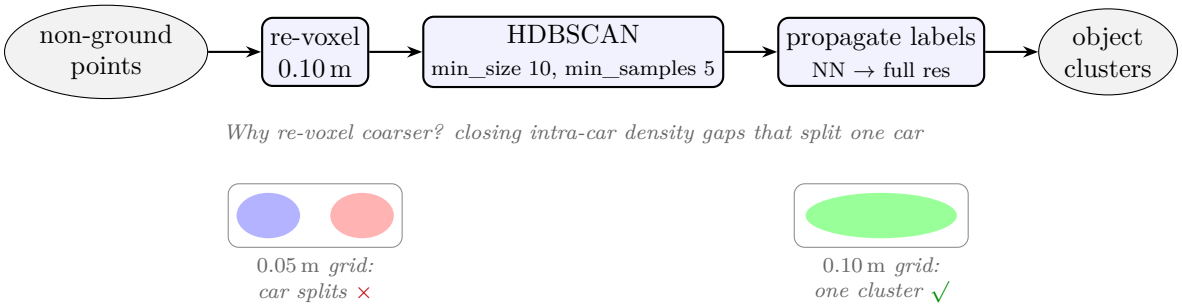


Figure 3.3: The clustering stage. Non-ground points are re-voxelised to a coarser 0.10 m grid, clustered with HDBSCAN, and the cluster labels propagated back to the full resolution points by nearest neighbour. Bottom: on the 0.05 m grid intra-car density gaps split one car (red), whereas the 0.10 m grid keeps it a single cluster (green).

3.2.5 Geometric Filtering of Non-Car Clusters

Input: object clusters | **Output:** car-candidate clusters (oriented boxes)

Most clusters are not cars: they are trees, poles, building facades, and fragments. A geometric filter rejects clusters whose oriented bounding box (OBB) is inconsistent with a car, using only cheap, interpretable shape gates (Figure 3.4). For each cluster the OBB is computed and the cluster is kept only if it passes all of the following:

- at least 15 points;
- OBB volume in $[0.5, 50]\,\mathrm{m}^3$;
- longest OBB extent ≤ 6.0 m, and the two largest extents above small floors (0.5 and 0.4 m), rejecting slivers;
- box centre no more than 1.5 m above the fitted ground plane, and vertical point span ≤ 1.8 m, rejecting tall structures;
- aspect ratio (longest / shortest extent) ≤ 6.0, rejecting thin walls and poles.

These bounds are generous car priors, not a tight car model, they are meant to remove obvious non-cars while leaving the fine car/not-car decision to the learned classifier. The

height gate is computed relative to the fitted ground plane, not to a fixed world height, so it is robust to the sensor not being perfectly level. Its individual gates are ablated in Chapter 4; the stage is high recall and low precision by construction, and the classifier is what turns its survivors into precise detections.

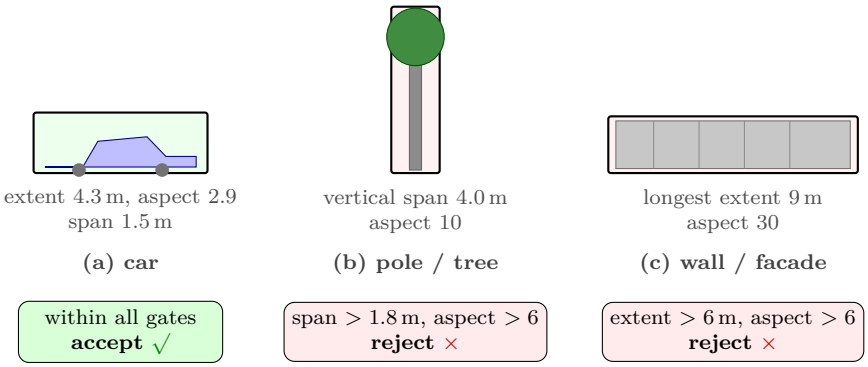


Figure 3.4: Geometric filtering by oriented-bounding-box gates (frozen priors, Table 3.1); a cluster survives only within every gate. (a) A car-sized box passes. (b) A pole or tree fails the vertical-span (1.8 m) and aspect (6) gates. (c) A wall or facade fails the longest-extent (6 m) and aspect gates.

3.2.6 The Car / Not-Car Point Classifier

Input: car-candidate clusters | **Output:** per-frame car / not-car label per cluster

Each surviving cluster is classified as car or not-car by a small dual branch PointNet. The binary decision replaces an earlier multi-class (car/bus/motorcycle) design that was dropped once the data showed near zero real bus and motorcycle instances, leaving the car only detector (Chapter 1).

Architecture: The network has two branches (Figure 3.5) and about 80 000 parameters, deliberately small:

- a point branch that takes 512 points (sampled or padded) normalised into the unit sphere, and applies shared per point MLPs ($3 \to 64 \to 128 \to 256$) followed by a max pool to a 256-d shape descriptor;
- a bounding box branch that takes an 8-dimensional metric feature vector (sorted OBB extents, volume, two aspect ratios, $\log(1 + \text{point count})$, vertical span), z-score normalised, through a small MLP to 32-d.

The two are concatenated and passed through a small head ($288 \to 128 \to 2$) with dropout. The bounding box branch is what re-injects absolute metric scale, which the unit sphere normalisation deliberately strips from the point branch: a distant small car and a nearby one have the same normalised shape but different real extents, and the metric branch is how the network sees the difference. There are no T-Nets, since clusters already sit in a consistent sensor frame.

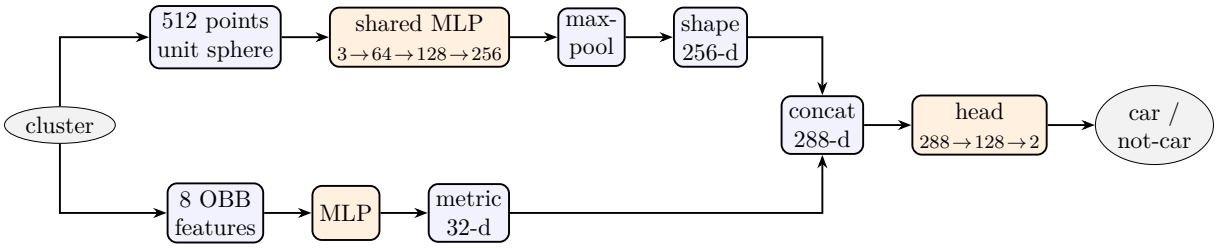


Figure 3.5: The dual branch classifier applied to one cluster. The point branch takes 512 unit sphere normalised points to a 256-d shape descriptor; the bounding box branch takes 8 metric shape features to 32-d. The two are concatenated and passed through a small head to a car/not-car decision. Learned blocks are shaded.

Training data (mined, not hand labelled): The classifier is trained on real LiDAR clusters mined automatically from SemanticKITTI. For each training frame the pipeline's own stages 1–5 are replayed, the ground truth semantic labels are propagated onto the surviving clusters by nearest neighbour, and each cluster is labelled car or not-car by majority vote, keeping only clusters whose majority class has purity ≥ 0.75. This yields 420 333 training clusters (sequences 00–07, 09, 10) and 130 394 validation clusters (sequence 08), with the intrinsic $\approx$21 %/79 % car/not-car imbalance handled by a class weighted loss. The key property is that the training clusters are produced by the same clustering and filtering the classifier meets at inference, including HDBSCAN splits and filter survivors, so it learns on exactly the distribution it must classify, at no manual annotation cost.

The from scratch production checkpoint: The production classifier is `stage_b_scratch_best.pth`, trained on the mined real clusters from random initialisation. An earlier two stage design first pretrained on synthetic ShapeNet partial renders ("Stage A") and then fine-tuned on the real clusters ("Stage B"). The two stage machinery is reported in Chapter 4 as an ablation: a controlled comparison there shows the synthetic pretraining to be redundant given the mined real clusters, so the final pipeline uses the simpler from scratch checkpoint. The synthetic pretraining, together with the finding that the sim-to-real gap is total and symmetric, is presented as a negative result rather than as a contribution.

Training: The from scratch checkpoint is trained for 15 epochs with Adam (learning rate 10^{-4}, weight decay 10^{-5}) and a step schedule that halves the rate every 10 epochs, at batch size 64. The loss is cross-entropy weighted by inverse square root class frequency (normalised to unit mean), which offsets the intrinsic $\approx$21 %/79 % car/not-car imbalance without resampling. Training clusters are augmented online with a random rotation about the vertical axis and Gaussian jitter ($\sigma = 0.005$ m); the input is the 512-point cluster of the architecture above. Runs are seeded and the reported checkpoint is the epoch with the best validation macro-F_1; that value, and the ablation comparison, are reported in Chapter 4.

3.2.7 Multi-Frame Tracking and Track Level Filtering

Input: per-frame car detections | **Output:** tracked cars (oriented boxes + stable track ids)

Detections are linked across frames by a centroid tracker. Each frame, current detections are matched to existing tracks by solving a Hungarian assignment on the pairwise centroid distance matrix; matches beyond 2.0 m are rejected, unmatched detections start new tracks, and a track that goes unmatched for more than 5 consecutive frames is retired. This is an offline, centroid only tracker, no motion model, no appearance, which is adequate here because detection, not association, is the bottleneck; an IoU-based (SORT-style) tracker is listed as future work.

The tracker also enables a track level filter that improves precision (Figure 3.6). Each detection carries the classifier's per frame label; a track is accepted as a car only if, across its lifetime, it collected at least 2 non-rejected votes, those known votes are a majority of its votes (≥ 0.5), and they agree on a single class. A cluster that the classifier flickers on (a car in one frame, not-car in the next) is thus resolved by temporal majority rather than trusted per frame. Because this filter consults the whole track, it is an offline post-processing step, consistent with the offline scope of the system; its contribution is isolated as an ablation row in Chapter 4. The tracker itself is computationally negligible.

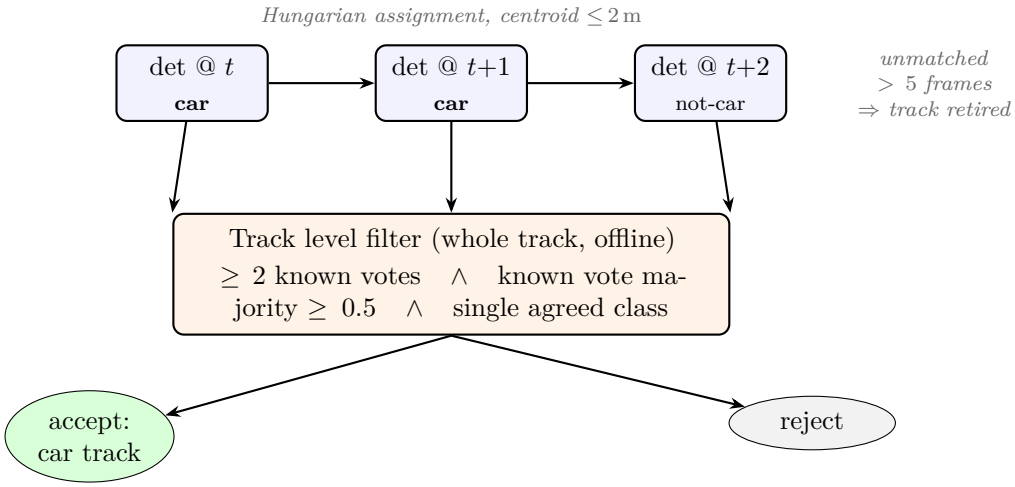


Figure 3.6: The tracker and track-level filter. Per-frame detections are linked into tracks by Hungarian assignment on centroid distance (matches beyond 2 m rejected), and the track-level filter resolves each track over its whole vote sequence rather than per frame (worked example: votes {car, car, not-car}). Both steps are offline.

3.2.8 The Frozen Pipeline Configuration

Table 3.1 lists the frozen parameter values that produce every number in this thesis. They are the committed `PIPELINE_CONFIG` and were not altered to obtain any reported result.

Table 3.1: Frozen pipeline configuration (committed `PIPELINE_CONFIG`); the classifier is the from-scratch checkpoint `stage_b_scratch_best.pth`.

Stage	Parameter	Value
Preprocess	z-filter threshold	-2.0 m
	voxel size	0.05 m
	denoise neighbours / std ratio	20 / 2.0
	voxel before denoise	true
Ground	RANSAC iterations	300
	inlier distance	0.2 m
	min. normal z	0.5
Clustering	method	HDBSCAN
	min. cluster size / min. samples	10 / 5
	clustering voxel size	0.10 m
Geometric	min. points	15
	volume range	$[0.5, 50]$ m^3
	max. longest extent	6.0 m
	max. height above ground	1.5 m
	max. vertical span	1.8 m
	max. aspect ratio	6.0
Classifier	model	dual branch PointNet (binary, from scratch)
	input points	512
Tracker	max. match distance	2.0 m
	max. disappeared frames	5
	min. track length	2
	min. known votes / ratio	2 / 0.5

The completion stage that operates on the tracked car detections (Figure 3.2, right) is the subject of Section 3.3; the evaluation of the detections themselves is Chapter 4.

3.3 Object Completion

This part of the method takes each detected car and completes it: from the one-sided, partial cluster a single LiDAR frame observes, it predicts the full car surface. The section presents the finished method in the order a cluster passes through it, the network, its training data, the inference time normalisation, the input gate, and the length geometry, and then, separately, gathers the negative results and lessons that shaped it (Section 3.3.8). Those false starts are of independent value; one is the motivating case for a project wide rule, so they are reported as findings rather than smoothed over. They are nonetheless kept out of the method description, so it reads as what the pipeline does rather than how it was arrived at. The engineering content here is not the network, a standard Point Completion Network (PCN) used off the shelf, but the data it is trained on and the geometry by which a real partial is presented to it. The evaluation of the finished method, on synthetic shapes and on real cars by independent classifier plausibility, is the subject of Section 4.3.1; here we describe how the completion is produced. Figure 3.7 gives the full completion path;

the subsections that follow describe each stage in turn.

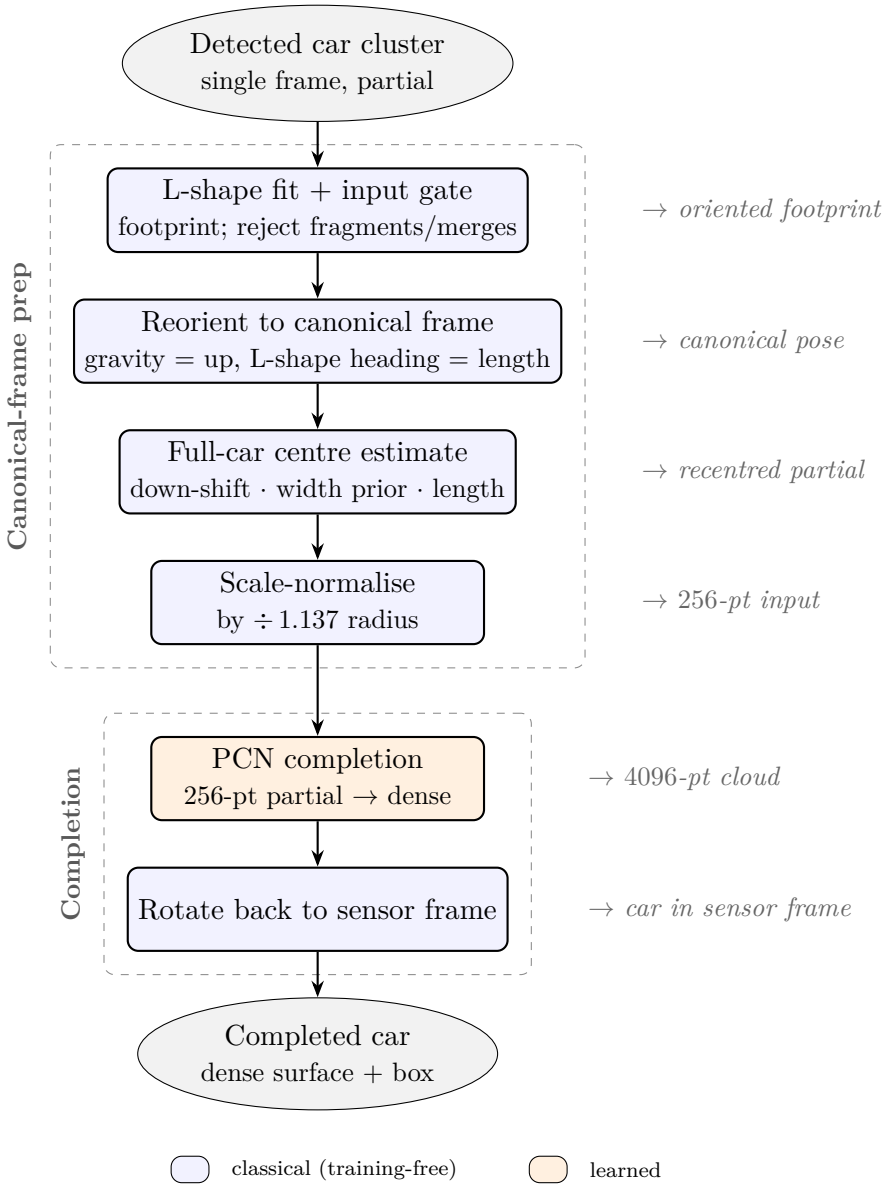


Figure 3.7: The object completion pipeline, grouped into two phases (dashed): a canonical-frame preparation phase and a completion phase, with each stage's output shown at right. The PCN is the only learned stage (shaded); every other step is inference-time geometry. Network internals are in Figure 3.8.

3.3.1 Completion Targets: Single Frame Car Clusters

Completion runs on the tracked car detections of Section 3.2. The input is a single frame cluster in the sensor frame, not the accumulated multi-frame track. This is a deliberate choice with an evidence basis. While temporal accumulation is a common technique, accumulating a moving car over many frames using a simple centroid tracker introduces significant motion smear and alignment errors. The slight centroid shifts frame-to-frame, when accumulated, stretch the car beyond its true boundaries and misalign the geometric features, producing an invalid completion target that no longer matches the true car shape. Therefore, a single frame provides one clean, if partial, observation. Whether completing

this partial produces a car-plausible shape on real data is what Section 4.3.1 measures; this section is how the completion is produced.

3.3.2 The Completion Network (PCN)

We use PCN [14], an encoder–decoder for point clouds (Figure 3.8). The encoder is a stacked PointNet: shared per point MLPs lift each point to a local feature, a max pool summarises the whole cloud into a global vector, that global vector is concatenated back onto every point, and a second MLP plus max pool stage produces a single 1024-dimensional shape embedding. The decoder is coarse-to-fine: a fully connected stage predicts 1024 coarse seed points, then a grid folding stage attaches a small 2D grid patch to each seed and learns to fold it into local surface geometry, densifying the coarse skeleton into the final cloud. Training minimises a two term Chamfer loss, $\mathrm{CD}_{\mathrm{coarse}} + 0.5\,\mathrm{CD}_{\mathrm{fine}}$; the fine term is down-weighted so the coarse skeleton is learned first. At inference the partial is subsampled to 256 points before it enters the encoder.

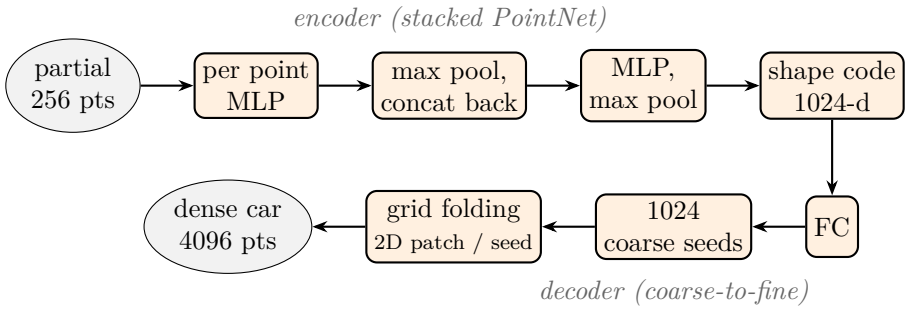


Figure 3.8: The completion network (PCN). A stacked PointNet encoder maps the 256-point partial to a 1024-d global shape code; a coarse-to-fine decoder predicts 1024 seed points and folds a small 2D grid patch onto each to produce the dense 4096-point car. Every block is learned; the architecture is used off the shelf and unmodified.

The architecture is not a contribution and was not modified; the controlled comparison against PoinTr that justifies keeping PCN, a small synthetic edge and equivalent real data behaviour, is reported with the completion ablations (Section 4.3.2). What makes PCN work on real cars is not the weights but the two subsections that follow: the distribution of partials it is trained on, and the frame a real partial is normalised into at inference.

3.3.3 Training Data: KITTI-Like Partials

PCN is trained on synthetic partials, and those partials must match the distribution of the real clusters it meets at inference or the network sees a different input at test time than it was trained on. A derisk study comparing candidate partial generators against real seq-08 clusters isolated three properties a naive generator gets wrong: it sampled a dense pinhole depth render, skipped the 0.05 m voxelisation and ground removal the real input has been through, and added range proportional noise (up to 0.15 m at 30 m) that washed out the scan ring banding real Velodyne data shows.

The generator used makes the synthetic partial match the real one: it orients the ShapeNet car to gravity, casts a single HDL-64E viewpoint (64 elevation beams, 0.09° horizontal resolution) from a sensor at 1.73 m and a sampled range of 8 m to 30 m, adds

small absolute noise (0.015 m), voxelises at 0.05 m to match the pipeline, and removes points within 0.30 m of the ground plane. The dense complete mesh sample remains the target. PCN retrained on these partials (`pcn_kitti_best.pth`) is in-distribution with the real input, and on a held out synthetic set produces clean cars rather than blobs; its synthetic Chamfer and F-score are reported in Chapter 4. Figure 3.9 contrasts the naive and KITTI-like partials for the same ShapeNet car.

Training: PCN is trained on the KITTI-like synthetic partials (input 256 points, target the dense mesh sample) for 100 epochs with Adam (learning rate 10^{-4}, weight decay 10^{-6}) and a step schedule that halves the rate every 40 epochs, at batch size 8, minimising the two term Chamfer loss above. No real cars enter training; the reported checkpoint (`pcn_kitti_best.pth`) is the best validation loss epoch. The subsequent geometry work is all inference time and does not retrain the network.

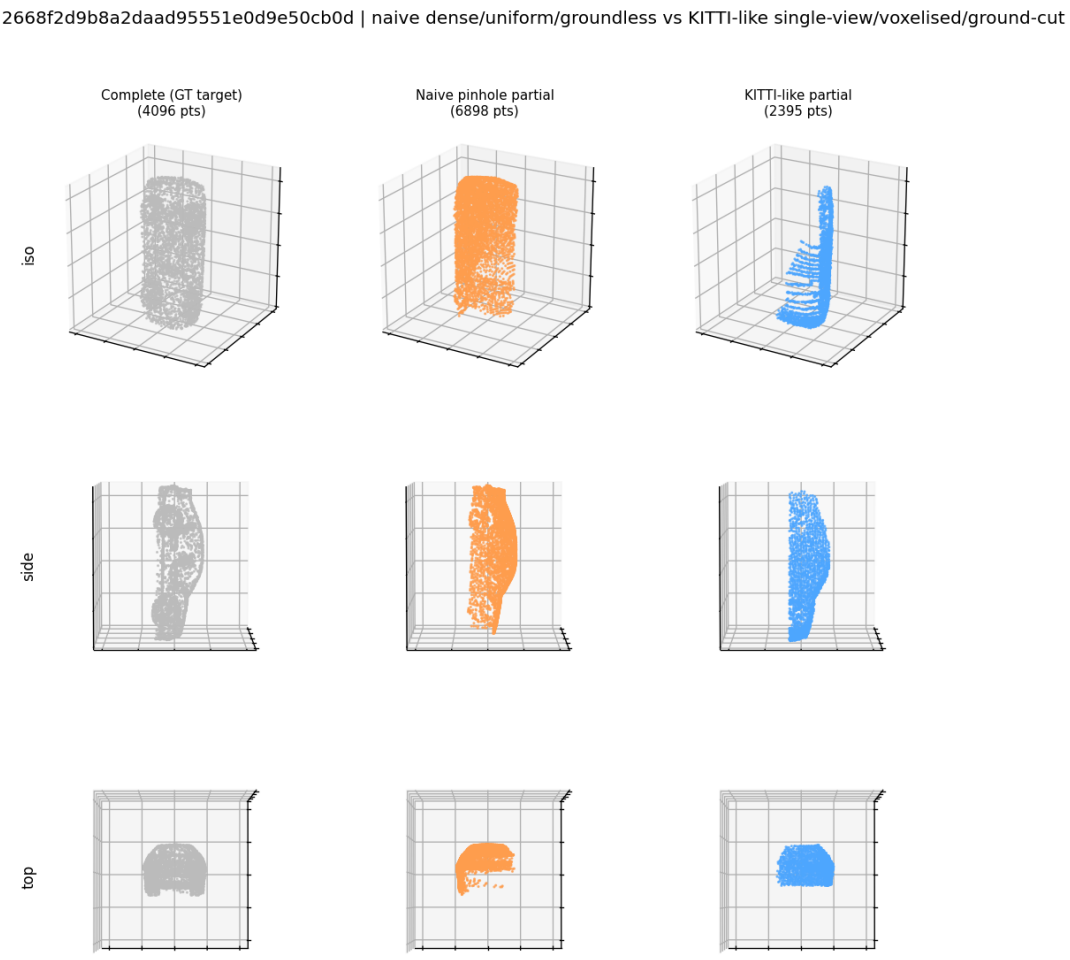


Figure 3.9: The training input distribution: one ShapeNet car with its complete surface target (left, grey) and two candidate training inputs, the naive pinhole partial (centre, orange) and the KITTI-like partial (right, blue) that reproduces real LiDAR (scan-ring banding, single-viewpoint coverage, 0.05 m voxelisation, ground cut). These are training inputs, not completion outputs.

3.3.4 Inference Time Partial Normalisation

Input: gated car cluster (sensor frame) + L-shape heading and per-car length | **Output:** 256-point partial in the canonical car frame (centred, scale-normalised)

PCN is trained on shapes expressed in a fixed canonical car frame, centred on the true car centre and scaled by the true car radius. At inference a real partial must therefore be mapped into that same frame before it is completed; anything else hands the network a normalisation it never saw during training. Reconstructing that frame from a one-sided partial is non-trivial, because a partial reveals neither the true centre nor the true radius directly. The inference path (`estimate_canonical_frame`) reconstructs it in three steps:

- Reorientation. The sensor frame is rotated into the canonical car frame (X = width, Y = up, Z = length): gravity maps to the up axis, and the horizontal heading axis maps to length. Heading comes from the L-shape footprint fit (Section 3.3.5), not a principal axis alignment.
- Full car centre estimate. A one-sided partial's bounding box centre is not the car centre, so the centre is corrected on each axis: shift down to undo the upward bias from the ground cut (the dominant correction, 0.25 m); push outward along width toward the occluded far side to a half car width prior (1.9 m); and push along length toward the occluded far end (Section 3.3.6).
- Scale. Normalise by the maximum point-to-centre distance divided by a fixed scale correction (1.137), the calibrated ratio between a one-sided partial radius and the true car radius.

With this path the same real seq-08 clusters recover the correct pose and scale; the ablation isolating this normalisation against a naive partial frame alternative, and the box overlap gains it produces, is reported with the completion ablations (Section 4.3.2). Viewed in the canonical car frame, the corrected completions are recognisably car-shaped: they recover the car footprint in bird's-eye view and a rough side silhouette (roofline included) even from very sparse input, a clean car footprint from a 64-point cluster in the extreme case. We are nonetheless careful about the strength of the claim: crispness varies with input density (cleaner on well-observed cars, blobbier on the sparsest), and a completion is a filled 4096-point cloud rather than a CAD clean surface. The sharpest completions are the in-distribution synthetic ones of Section 3.3.3; on real data the normalisation's value is corrected geometry, pose and scale, and the car plausibility of the completed shape that Section 4.3.1 reports. A calibration study on synthetic cars further identified the residual dominant error as centre estimation, not scale, which is what motivates the geometry work in the rest of this section.

3.3.5 L-Shape Gating of Completion Inputs

Input: detected car cluster (single-frame partial) | **Output:** fitted footprint + accept/reject decision (fragments and merges rejected)

Even with correct normalisation, some clusters cannot complete into a plausible car because the input is not one car: HDBSCAN fragments (part of a car) and merges (two cars, or a 90° heading error) both feed the completion network implausible input. To detect them we fit the cluster's ground plane footprint with a search-based L-shape fit [31],

which scores edge adherence over candidate yaw angles and stays well-defined on near square footprints where PCA degenerates. The fitted footprint gives an input quality gate: a cluster is rejected as a fragment if its fitted length is below 2.7 m, or as a merge if its fitted width exceeds 2.3 m (Figure 3.10), so the gate keeps implausible inputs out of the completion network rather than relying on it to fix them.

On seq-08 every implausible completion originated from a fragment or merge input, so the gate's effect on completion quality is a clean ablation, reported in Section 4.3.2. An honest secondary finding from the same study: the L-shape heading itself is neutral against plain PCA on real data, the fit earns its place by detecting bad inputs, not by giving a better heading.

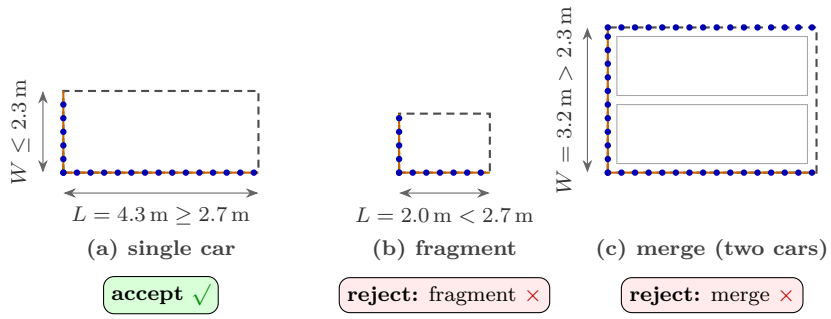


Figure 3.10: L-shape gating of completion inputs. Each cluster's ground-plane footprint (blue points) is fit with a search-based L-shape (orange edges), and the fitted rectangle (dashed) yields a length and a width. (a) A single car clears both gates ($L \geq 2.7$ m, $W \leq 2.3$ m). (b) A fragment is too short ($L < 2.7$ m). (c) A merge is over-wide ($W > 2.3$ m).

3.3.6 Per Car Length Estimation

Input: the track's gate-passed footprint length fits | **Output:** per-car length estimate (feeds the centre estimate)

The canonical frame centre estimate re-centres the occluded lateral side and the ground cut, but a partial truncated at the occluded far end is normalised around its observed near portion, so PCN's output tends to stop short of the far end, the least covered region of a one-sided car. Two successive fixes address it, both inference only (no retraining).

The fixed length prior: A longitudinal, extend only push mirrors the width prior along the length axis: the centre is pushed toward the occluded far end so the completion reaches an assumed full car length of 4.14 m, a population median car length. A larger 4.5 m prior was rejected because it over-extended genuinely compact cars.

The per car estimate: A single 4.14 m prior is a population median, so it is wrong in both tails, it over-extends compact cars and under-extends long ones. It is replaced by a per car length, `track_length_estimate`: the 90th percentile of the car's own gate passed footprint length fits, plus a 0.12 m bias correction, falling back to the fixed prior when a track has fewer than 5 gate passed frames. The aggregate is a quantile, not the maximum, because a single contaminated L-shape fit would set the maximum. This is a per track quantity, so the pipeline (which owns the track) computes it and passes it in; the completion module itself stays stateless.

Limitation: The fallback is not rare: a non-trivial fraction of tracks have fewer than five gate passed frames and fall back to the fixed prior. This is stated as a live limitation, not a solved problem; the exact fallback rate on sequence 08 is reported with the completion results, and it is revisited in the discussion.

3.3.7 The End-to-End Completion Path

Input: single-frame car cluster (sensor frame) | **Output:** completed dense car (4096-point surface + box, sensor frame)

To summarise, a single frame car cluster is completed by: fitting its footprint and rejecting fragments and merges (L-shape gate); reorienting it into the canonical car frame with no PCA; estimating the full car centre (down-shift for the ground cut, width prior for the occluded side, per car length estimate for the occluded far end); normalising by the scale corrected radius; running the KITTI-like-trained PCN; and rotating the output back to the sensor frame. Every step is inference time geometry except the one-off retraining on KITTI-like partials; no step fine-tunes on the evaluation cars.

Figure 3.11 shows what this path produces on real seq-08 cars across the input density range, in the canonical car frame. The completed cloud (green) fills the car footprint and a rough side silhouette that the single frame partial (blue) leaves one-sided, recognisably car-shaped down to a 64-point input. Consistent with the caveats above, the sparsest inputs complete more coarsely and can inflate the width, and the clouds are filled rather than CAD clean. The point here is qualitative shape; the quantitative real data assessment, whether the completed cloud reads as a car to an independent classifier, is what Section 4.3.1 sets out to measure.

3.3.8 Negative Results and Lessons

The method above was not reached in one step. Two false starts shaped it, and each carries a lesson worth stating in its own right rather than smoothing over. They are gathered here so that the method description reads as what the pipeline does. A third intervention that was tried and rejected, a length dependent far end compensation, is reported as a completion ablation (Section 4.3.2).

The blobs were not a synthetic data problem: The first attempts to run PCN on real cars all produced "blobs", rounded, car-sized clouds with no recognisable car structure. This was initially attributed to the training data being synthetic, and the response was a series of sim-to-real remedies: LiDAR noise augmentation of the synthetic partials, and real data fine-tuning. None of them fixed the blobs, and the fine-tuning attempt is recorded as a negative result, on this data, fine-tuning PCN on mined real pairs did not recover real car geometry. The blobs had two causes, and neither was the network weights: the training input distribution (Section 3.3.3) and the inference normalisation (Section 3.3.4), the two method steps above. The lesson is to suspect the data and the input pipeline before the model.

Train and inference semantics must match: Even after retraining on KITTI-like partials, real clusters still blobbed, and the cause is the sharper lesson. An earlier inference path normalised each real partial by a 3D PCA frame and the partial's own centroid and radius, a normalisation the model had never seen, since it was trained in the canonical car

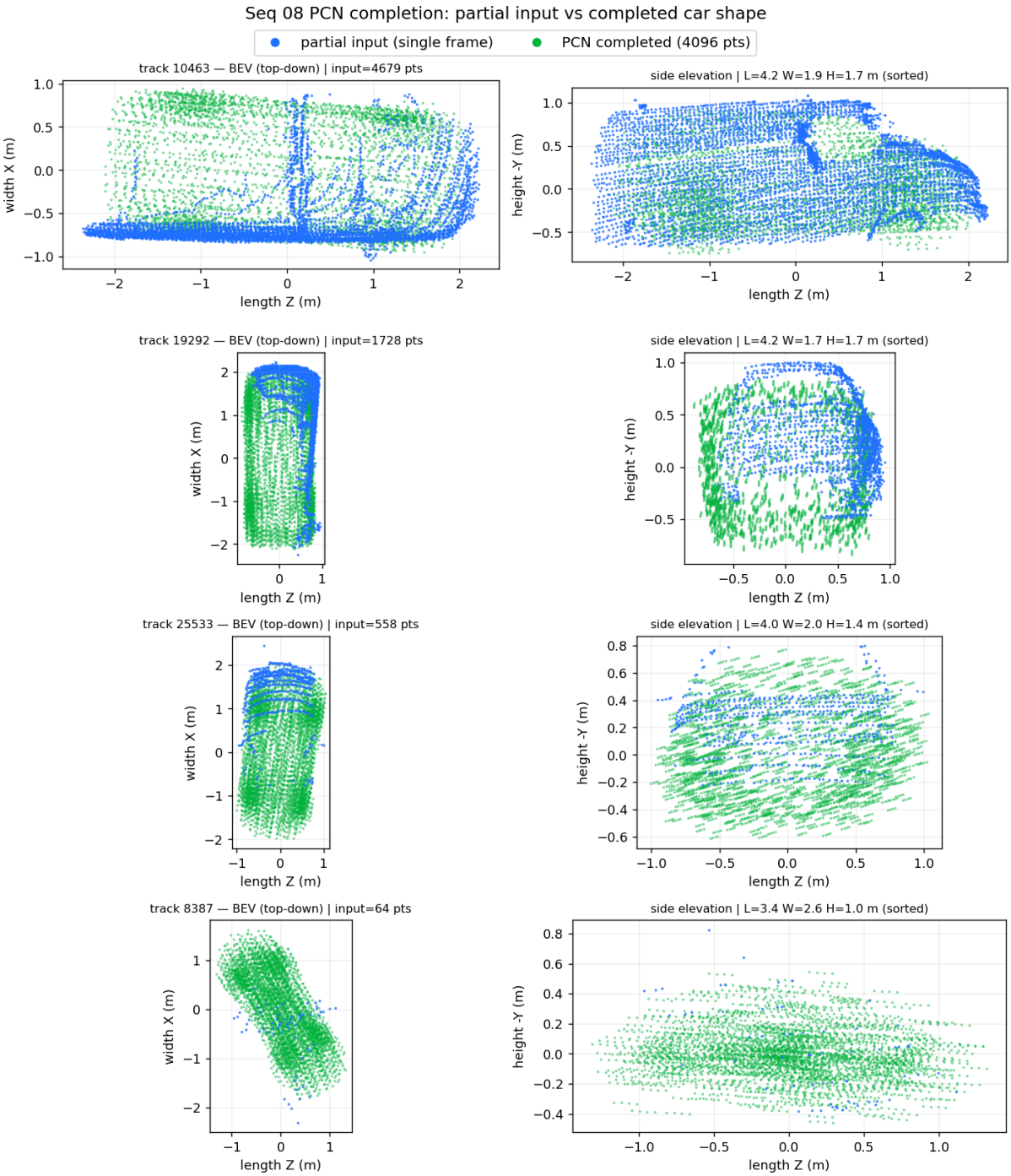


Figure 3.11: The final completion path on real cars, four tracks spanning the input density range, each in the canonical car frame: bird's-eye view (left of each pair) and side elevation (right). Blue is the single-frame partial input, green the PCN completion.

frame of Section 3.3.4. This silent train/inference mismatch alone produced blobs, and it did so for every checkpoint, including in-distribution synthetic ones where the model is otherwise clean. Because the two frames each looked internally reasonable, the divergence was invisible until measured. This incident is the motivating example for the project's rule that any external integration or reuse gets a written semantics map , coordinate frame, centring, and scale conventions, reconciled before code is run.

3.4 Summary and Methodological Limitations

This chapter has set out the full method: the modular detection pipeline that turns a raw scan into car detections, and the single frame completion path with its input gate and length prior. Several limitations are built into these choices and are flagged here so the evaluation is read with them in view; each is taken up in full in the discussion of Chapter 5.

- Ground model: a single global RANSAC plane assumes near planar ground; it holds on the flat urban sequences used here but would degrade on strong slopes, where a grid-based model (future work) would be needed (Section 3.2.3).
- Offline by design: the tracker and track level filter consult the whole sequence, and the pipeline runs far below sensor rate; real-time operation is out of scope (Section 3.2.1).
- Single frame, static car completion: completion runs on one frame's cluster; moving cars are completed but are considered separately, and no completion is scored against an accumulated multi-frame shape (Section 3.3.1).
- Length prior fallback: the per car length estimate falls back to the fixed 4.14 m prior on tracks with fewer than five gate passed frames, and a residual far end under-extension on long cars is a characterised, not eliminated, limit (Section 3.3.6).
- Synthetic only completion training: PCN sees only synthetic KITTI-like partials; real data fine-tuning was a negative result, so a sim-to-real residual remains (Section 3.3.8).

With these caveats stated, Chapter 4 puts each component to the test: the detection operating point and its ablations, the recall root cause, and the completion results (synthetic fidelity and independent classifier plausibility). Chapter 5 then returns to the limitations above, alongside the evaluation questions the results raise.

Chapter 4

Experimental Evaluation

This chapter evaluates the full pipeline end to end. It first fixes the experimental setup, dataset, the pipeline under test, metrics, baselines, and test environment (Section 4.1). It then evaluates the two halves of the system in turn. The detection evaluation (Section 4.2) reports the operating point and its cross-validated generalisation, the stage ablations, an analysis of where detection succeeds and fails, and runtime. The completion evaluation (Section 4.3) reports the completion results, synthetic fidelity, independent classifier plausibility on real cars, and reference free fidelity, together with the completion ablations. The recall analysis in the detection half root causes the recall limit to the same clustering step that governs input cleanliness for completion, the link between the chapter's two halves.

4.1 Dataset & Protocol

The framework is grounded on two public datasets, matching its two halves. SemanticKITTI [2] supplies the real Velodyne HDL-64E scans used for detection, weak label mining, and evaluation; its per point semantic and instance labels serve as ground truth. ShapeNetCore v2 [15] supplies the synthetic complete car meshes from which the completion network's training partials are rendered (Section 3.3.3); no real car is ever hand labelled for completion. Detection and completion therefore draw on real and synthetic data respectively, and the framework requires zero additional manual annotation: the classifier's training data is auto-mined from existing semantic annotations and the completion network is trained exclusively on synthetic shapes.

We evaluate on the eleven labelled SemanticKITTI odometry sequences (0 to 10), whose per point semantic and instance labels serve as ground truth. The official test sequences 11 to 21 carry no public labels, so no ground truth detection metric can be computed on them. This is a property of the dataset, not a choice, and it is why no labelled sequence can be reserved as a conventional held out test set. Before the per sequence split, Tables 4.1 and 4.2 fix the two datasets' roles and the Velodyne HDL-64E that produces the real scans; its 64-beam geometry and strong range fall-off set the point cloud sparsity and density variation the clustering stage must absorb. Table 4.3 then summarises the labelled sequences, their frame counts, and their role in each half of the framework.

Table 4.1: Dataset roles and modalities: SemanticKITTI (real scans; detection, mining, and evaluation) and ShapeNetCore v2 (synthetic meshes; completion-network training).

Specification	SemanticKITTI	ShapeNetCore v2
Data Modality	Real LiDAR Scans	Synthetic CAD Meshes
Framework Role	Detection, Mining, & Evaluation	Completion Network Training
Total Volume	23 201 frames (11 sequences)	∼3150 models (Car category)
Annotations Used	Point-wise Semantic & Instance	None (Synthetic partials)

Table 4.2: SemanticKITTI sensor specification: the Velodyne HDL-64E that produces every labelled scan.

Parameter	Specification
Sensor Model	Velodyne HDL-64E
Vertical Channels	64 beams
Frame Rate (Spin)	10 Hz
Horizontal Resolution	∼0.09°
Max Range	120 m
Points per Frame	∼100 000 to 120 000

Table 4.3: The eleven labelled SemanticKITTI sequences (0 to 10; 23 201 frames total), with per sequence frame counts and roles. *Classifier role*: the headline detection split (train / validation = seq 08 / train). *PCN role*: every sequence is completion evaluation only, since PCN trains on synthetic ShapeNet.

Seq	00	01	02	03	04	05	06	07	08	09	10
Frames	4541	1101	4661	801	271	2761	1101	1101	4071	1591	1201
Classifier role	train								val	train	
PCN role	completion evaluation only (PCN trained on synthetic ShapeNet)										

Train, validation, and test split: For classification, the binary car/ not-car classifier is trained on the mined clusters of sequences 0 to 7, 09, and 10 and validated on sequence 08; the leakage controlled result is the LOSO cross-validation across all eleven sequences (Section 4.2.1). For completion, the PCN network is trained on synthetic KITTI-like ShapeNet car partials, with no real data split, and is evaluated on all eleven labelled sequences through the independent classifier plausibility protocol (Section 4.3.1). The SemanticKITTI test sequences (11 to 21) are unusable for either, as they carry no public labels.

Classifier evaluation uses leave-one-sequence-out cross-validation: Rather than report detection on the sequence the classifier was tuned against, we evaluate the binary classifier under leave-one-sequence-out (LOSO) cross-validation over all eleven sequences. For each held out sequence K a classifier is trained on the other ten sequences and then

run on the full sequence K; the held out sequence contributes zero clusters to that fold's training data and is never used to select the checkpoint. Checkpoint selection is removed entirely: every fold trains for a fixed 15-epoch budget and the last epoch checkpoint is used (historically the best epoch was 13/15, so last $\approx$ best), so the held out sequence is untouched during both training and model selection. In the per fold table below, therefore, the classifier scoring sequence K has never seen sequence K during training or model selection. Because the Stage B clusters are stored with a sequence prefix in their filenames, the folds are a re-partition of the existing mined data and require no re-mining.

Two sequences carry additional, distinct roles:

- Sequence 08 (reported operating point; 4071 frames). The largest labelled sequence is the operating point at which the pipeline is reported in detail: the stage ablations (Section 4.2.2), the recall root cause analysis (Section 4.2.3), and the IoU threshold sensitivity below are all computed on the sequence-08 output of the final classifier, and the completion constants were tuned on it (the completion evaluation itself, Section 4.3.1, spans all eleven sequences). Sequence 08 was never used to tune the classical stages. The final classifier was historically selected on sequence 08, so its sequence-08 numbers could in principle be optimistic; the LOSO fold-08 classifier (trained on 0 to $7, 9$ to 10, last epoch, never selected on 08) settles this empirically below.
- Sequence 00 (classifier training split, tuning baseline). Sequence 00's classical pipeline parameters (`PIPELINE_CONFIG`) were historically hand tuned on it, and it also sits in the classifier's training split, so its detection recall reads optimistically; we isolate that residual in the LOSO aggregate below (dropping fold-00 moves pooled recall by 0.022). This does not touch the completion evaluation, which is a paired raw-versus-completed comparison on identical inputs and whose network never saw any real sequence.

Ground truth targets and the eligibility rule: The pipeline is a car detector, so the evaluation counts only the two car-bearing SemanticKITTI classes as valid ground truth objects: car (class 10) and moving car (class 252); we refer to this target set as supported vehicles. A ground truth car instance is a maximal set of points sharing one instance id within these classes.

An instance is counted in the evaluation only if at least 10 of its points survive preprocessing, the z-filter, voxel downsampling, statistical denoising, and ground removal that every scan passes through before clustering. This is the recall denominator: recall is computed against ground truth car instances with ≥ 10 points surviving preprocessing, not against every annotated car. The rule exists because an instance reduced to a handful of points by occlusion or range cannot be segmented by any clustering method, so including it would measure sensor sparsity rather than the pipeline.

This choice narrows the denominator, and it is important to quantify by how much. On a stride-20 sample of sequence 08 (204 of 4071 frames, drawn uniformly across the drive), of all annotated car instances (2332 pooled), 25.5% are excluded by the surviving points rule. Measured instead against instances that have ≥ 10 points in the raw scan (2033 pooled), 14.6% are excluded by preprocessing.[1] Because an excluded instance can never be

[1]These are two distinct ≥ 10-point rules: one thresholds raw label points (a distinct car count), the other thresholds points surviving preprocessing (the eval denominator). They are not interchangeable.

matched, the sequence-08 operating point recall of 0.732 against the eligible denominator corresponds, by inference, to a recall of approximately 0.55 against all annotated cars. (The eligible-to-annotated survival ratio on this sample is $1737/2332 = 0.745$; the ratio is a stride-20 estimate, so this figure is inferred, not separately measured.) We report the eligible denominator recall as the headline and disclose the all annotated figure alongside it.

Every quantitative claim about detection in this thesis is produced by a single evaluation script under one fixed protocol. Because the results, ablation, and recall analysis sections all report numbers from this protocol, we state it in full here and refer back to it rather than restating it per table. The protocol was frozen before the write up began; no configuration, checkpoint, or metric definition described below was altered to produce a reported number.

4.1.1 Baseline Methods

Detection is not compared head-to-head against the supervised networks of Section 2.1.5: those report different metrics under different protocols, and Table 2.2 sets out that non-comparison and the paradigm gap behind it. Within this pipeline the classical stages and the track filter are instead compared against one another by ablation (Section 4.2.2). Completion is judged against explicit references on the same inputs: the raw partial (the paired baseline the completion must beat), a dense static accumulation ceiling (the shape quality upper bound of a correctly shaped car), and, for architecture, PoinTr. Real data completion quality is scored by the independent classifier plausibility of Section 4.3.1, with the standard reference free metrics UCD and UHD (Section 2.1.4) reported alongside it in Section 4.3.1.

4.1.2 Evaluation Metrics

Detections and ground truth instances are compared at the point level. For a detection mask D and an instance mask G over the surviving points, the overlap is the intersection over union

$$\mathrm{IoU}(D, G) \;=\; \frac{|D \cap G|}{|D \cup G|}.$$

All detection–instance pairs with IoU $\geq \tau$ are collected, sorted by descending IoU, and assigned greedily and one-to-one: the highest IoU pair is matched first, and once a detection or an instance is used it cannot be matched again. This yields, per frame:

- a true positive (TP) for each matched pair,
- a false positive (FP) for each unmatched detection,
- a false negative (FN) for each unmatched eligible instance.

Precision, recall, and F_1 are then computed from TP, FP, and FN micro-averaged over the sequence: counts are pooled across all frames and the ratios taken once, so denser frames contribute proportionally more. We also report the mean IoU of matched detections, the mean of the IoU values of the accepted TP pairs only. This is a quality measure of the matches that were made; it says nothing about missed cars and must not be read as an overall segmentation score.

Track level filtering is enabled by default for the headline numbers: an offline centroid tracker links detections across frames, and only tracks meeting a minimum length and

class consistency requirement are accepted. This is a post-hoc, offline step (it uses the whole sequence), consistent with the offline scope of the system; its effect is isolated as an ablation row in the ablation section (Section 4.2.2).

Choice of IoU threshold and its sensitivity: All headline numbers use $\tau = 0.3$. A point level threshold of 0.3 is permissive relative to the bounding box 0.5 common in detection benchmarks, so it is fair to ask whether the results are an artifact of a lenient gate. They are not, and the reason is visible in Table 4.6: the mean IoU of matched detections is 0.913 on sequence 08, far above either threshold, so the matches are not marginal overlaps sitting just past τ.

To make this concrete we re-ran the full sequence 08 evaluation at $\tau \in \{0.25, 0.30, 0.50\}$ with everything else frozen (Table 4.4).

Table 4.4: IoU threshold sensitivity on the full sequence (frozen configuration); the $\tau = 0.30$ row is the headline. Mean IoU is over matched detections.

τ	Precision	Recall	F_1	Mean IoU	TP / FP / FN
0.25	0.914	0.735	0.815	0.910	25 660 / 2421 / 9262
0.30	0.911	0.732	0.812	0.913	25 568 / 2513 / 9354
0.50	0.884	0.711	0.788	0.927	24 837 / 3244 / 10 085

F_1 falls only 2.4 points from the headline $\tau = 0.30$ to the strict $\tau = 0.50$ (0.812 to 0.788), and 2.7 points across the full tested range from the permissive $\tau = 0.25$ (0.815). The reported quality is therefore stable under the choice of threshold and does not depend on the lenient value. The mean IoU of matched detections rises slightly at the stricter gate, which keeps only the better overlapping matches. We had pre-registered that a threshold sensitive result would be reported as a finding rather than tuned away; it was not sensitive.

4.1.3 Implementation Details

The system under evaluation is the frozen detection and completion pipeline of Chapter 3, run unmodified by a single script: ground removal, HDBSCAN clustering, the geometric filter, the binary classifier, and the centroid tracker with its track level filter for detection; and, for each detected car, the KITTI-like-trained PCN behind the L-shape input gate for completion. Every number below uses the committed `PIPELINE_CONFIG` (Table 3.1); no stage, checkpoint, or parameter was changed to obtain a reported result.

The framework's defining property is that supervision is confined to two learned stages, with all geometric reasoning done by training free classical methods (Table 4.5): the classical stages need no training data at all, the classifier is trained on supervision auto-mined from existing semantic annotations, and the completion network on synthetic ShapeNet partials.

Table 4.5: The framework's stages by supervision: six unsupervised classical stages and two supervised (learned) stages that use no additional manual annotation.

Unsupervised (classical, training free)	Supervised (learned, no real labels)
z-filter; denoise + voxel downsample; RANSAC ground removal; HDBSCAN clustering; geometric OBB filter; centroid tracker + track level filter	Binary car/not-car PointNet classifier (mined weak labels); PCN completion (synthetic ShapeNet partials)

All runs are deterministic: ground plane RANSAC and the classifier use fixed seeds (Chapter 3), so a rerun reproduces every reported number. Timings are wall clock on an AMD Ryzen 7 7800X3D with an RTX 3070 Ti, measured on the full sequence 08 under the promoted configuration (Section 4.2.4). The classifier and PCN training hyperparameters, optimiser, learning rate schedule, epochs, and batch size, are given with each model in Chapter 3 and are not repeated here.

4.2 Detection Evaluation

This section evaluates the detection half of the pipeline: the reported operating point and its cross-validated generalisation (Section 4.2.1), the ablations that isolate each stage's contribution (Section 4.2.2), an analysis of where detection succeeds and fails (Section 4.2.3), and the per frame runtime (Section 4.2.4).

4.2.1 Detection and Classification Results

The Reported Operating Point on Sequence 08

Table 4.6 gives the reported operating point on the full sequence 08 under this protocol, produced by the final classifier (the one the completion pipeline actually runs with). This is the operating point the ablation and recall analysis sections of this chapter decompose; the cross-validation that confirms it follows in Section 4.2.1.

Table 4.6: Reported detection operating point on the full sequence 08 (frozen protocol, final classifier), with its leave-one-sequence-out held-out counterpart (fold-08, a classifier blind to sequence 08; Section 4.2.1). Mean IoU is over matched detections only.

Classifier (seq 08)	Precision	Recall	F_1	Mean IoU	TP / FP / FN
Reported (final)	0.911	0.732	0.812	0.913	25 568 / 2513 / 9354
Held out (LOSO fold-08)	0.888	0.736	0.805	0.913	25 715 / 3228 / 9207

The pipeline is precision saturated and recall limited: false positives are flat at roughly one per frame, while recall is set by how many eligible cars the clustering stage recovers as single objects. The remaining sections of this chapter report the ablations that decompose this operating point, and Section 4.2.3 root causes the recall shortfall.

Per Frame Detection Results

The headline operating point is the reported row of Table 4.6 (Section 4.1): precision 0.911, recall 0.732, F_1 0.812, and a mean IoU of matched detections of 0.913, over

25 568 / 2513 / 9354 TP / FP / FN. Two properties of this operating point organise the rest of the chapter, and both are visible directly in the per frame record rather than only in the pooled totals.

First, the pipeline is precision saturated. Across the full sequence the false positive count is low and stable, about 0.6 per frame (2513 over 4071 frames), independent of how many cars are in view. Figure 4.1 shows this spatially, detections track the true parked and moving cars along the street, with few spurious boxes, and Figure 4.2 shows it over time: the FP trace is low and stationary while the number of ground truth cars per frame swings widely.

Second, the pipeline is recall limited, and the limit is structural. The recall shortfall traces to a measured clustering failure: HDBSCAN splits large, close cars into fragments that no longer match a single instance, and the split rate is highest exactly where cars are densest and nearest (53.5 % at 0 m to 10 m). Section 4.2.1 exhibits the resulting near range recall dip directly, and the time series (Figure 4.2) shows per frame recall tending to fall in the denser frames while false positives stay flat. We root cause the splitting mechanism in Section 4.2.3; here we establish only that recall, not precision, is the dominant error mode, and that it does not come from the classifier misfiring (false positives stay flat regardless).

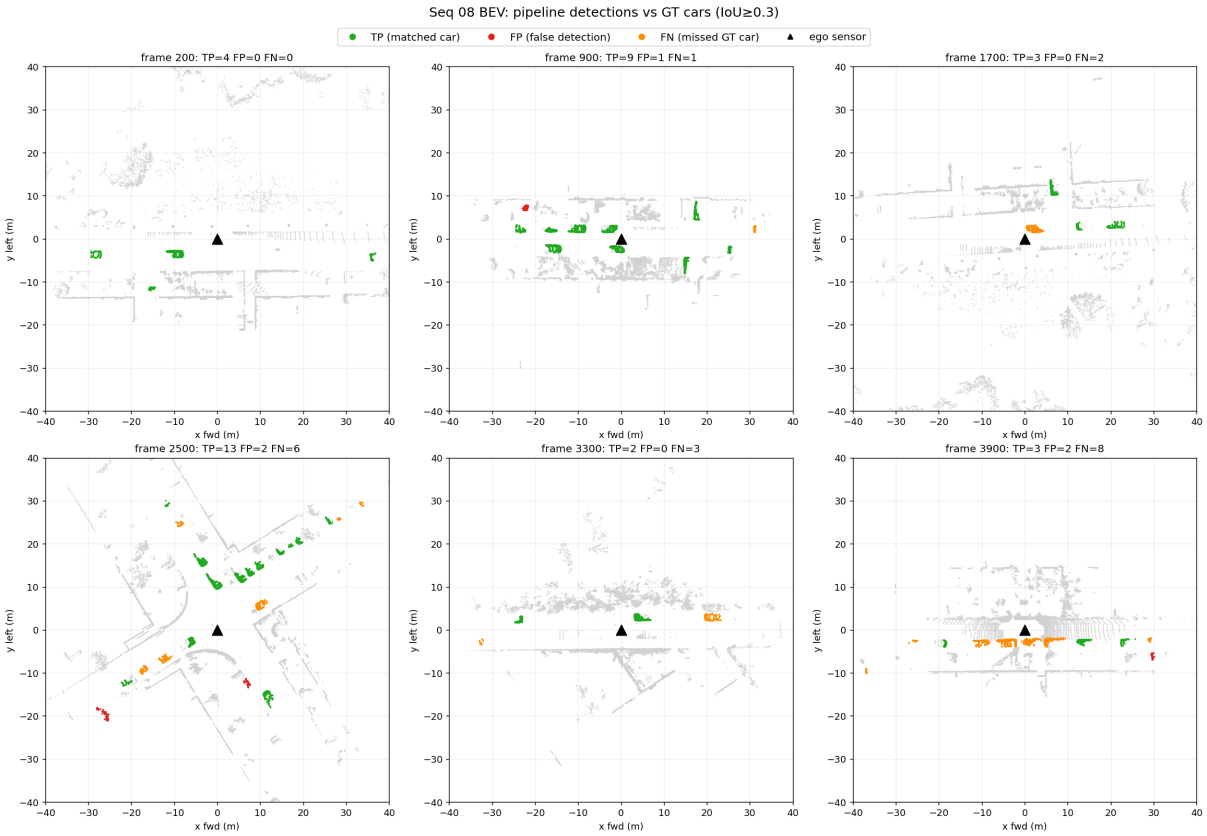


Figure 4.1: Bird's-eye view of detections on a representative frame: detected car clusters (coloured) overlaid on the ground-removed point cloud, with ego at the origin.

Leave-One-Sequence-Out Cross-Validation

The operating point above is reported on sequence 08, which is also the sequence the final classifier was historically selected on. To show that this selection does not inflate the

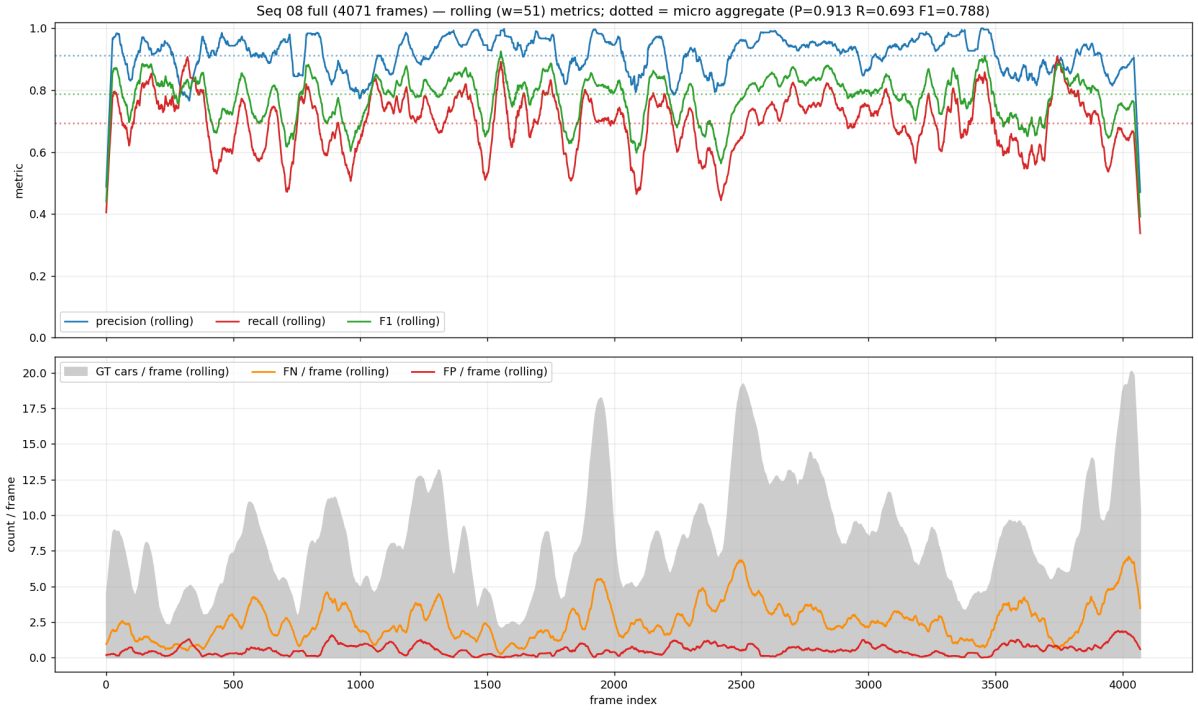


Figure 4.2: Per-frame detection behaviour over a full sequence: the false-positive count (lower trace) and per-frame recall against the number of ground-truth cars per frame.

result, and to characterise the per sequence recall spread that a single reporting sequence cannot, we ran the LOSO protocol of Section 4.1: eleven folds, each training a classifier on ten sequences and evaluating the full held out eleventh, so every fold's classifier is blind to the sequence it scores. Table 4.7 reports all eleven folds, the pooled micro-average (counts summed across folds, comparable to the single operating point number), and the macro mean ± standard deviation across folds.

Table 4.7: Leave-one-sequence-out cross-validation over all eleven labelled sequences (full sequences, $\tau = 0.3$, final configuration). "Pooled" sums TP/FP/FN across folds; "excl. seq 00" drops the seq-00 fold (Section 4.1).

Held out seq	Precision	Recall	F_1	Mean IoU	TP / FP / FN
00 (4541 fr.)	0.921	0.758	0.831	0.933	42 583 / 3675 / 13 607
01 (1101 fr.)	0.957	0.336	0.498	0.953	540 / 24 / 1066
02 (4661 fr.)	0.823	0.701	0.757	0.904	10 941 / 2345 / 4663
03 (801 fr.)	0.796	0.623	0.699	0.905	1528 / 391 / 925
04 (271 fr.)	0.824	0.510	0.630	0.836	539 / 115 / 517
05 (2761 fr.)	0.880	0.725	0.795	0.910	10 585 / 1448 / 4013
06 (1101 fr.)	0.873	0.637	0.737	0.906	6379 / 931 / 3632
07 (1101 fr.)	0.916	0.726	0.810	0.930	11 089 / 1011 / 4179
08 (4071 fr.)	0.888	0.736	0.805	0.913	25 715 / 3228 / 9207
09 (1591 fr.)	0.851	0.638	0.729	0.913	5072 / 888 / 2882
10 (1201 fr.)	0.745	0.557	0.638	0.872	1826 / 625 / 1450
Pooled (all 11)	**0.888**	**0.717**	**0.793**	**0.919**	116 797 / 14 681 / 46 141
Pooled (excl. seq 00)	0.871	0.695	0.773	0.912	74 214 / 11 006 / 32 534
Macro mean ± std	0.861±0.059	0.632±0.119	0.721±0.095	0.907±0.030	,

Three things follow. First, model selection did not inflate the reported recall. The fold-08 classifier, blind to sequence 08, reaches recall 0.736 against the sequence-08 operating point's 0.732, if anything marginally higher, not lower, so the recall headline is not a product of having selected the classifier on the reporting sequence. The model selection optimism that does exist is confined to precision and is small: fold-08 precision is 0.888 versus the final 0.911 (an extra 715 false positives across 4071 frames), exactly the two point, precision only exposure the stage decomposition predicts (Section 5.2.2). The two sequence-08 rows are not two separate experiments but one classifier under two checkpoint selection rules. The final model and the fold-08 model share the same architecture, the same ten training sequences (0 to 7, 9 to 10), and the same from scratch 15-epoch regime, and differ only in which epoch is frozen. The final model takes the epoch that scored best on the sequence-08 validation set; the fold-08 model takes the last epoch and never inspects sequence 08 at all. The $0.732 \rightarrow 0.736$ (recall) and $0.911 \rightarrow 0.888$ (precision) deltas therefore isolate the effect of that selection: selecting on sequence 08 bought about two precision points and no recall. The isolation is not perfect, because sequence 08 cannot serve as the fold's loss monitoring split: the fold-08 model trains on 95 % of the final model's clusters, with a 5-percent monitoring slice set aside. That confound removes training data from the fold-08 model, which would depress its recall rather than inflate it, so it only strengthens the "recall was not inflated" reading. Second, the pooled cross-validated operating point over all eleven sequences, precision 0.888, recall 0.717, F_1 0.793, mean IoU 0.919, sits within roughly two points of the sequence-08 operating point on every metric. The reported numbers are therefore representative of the labelled set and not specific to sequence 08. Third, the per sequence spread is now explicit: recall ranges from 0.336 on sequence 01 (a sparse highway drive) to 0.758 on sequence 00 (dense urban), with a macro mean of 0.632 ± 0.119; precision is far steadier (0.861 ± 0.059). The gap between the pooled recall (0.717) and the macro mean (0.632) is driven by the smallest, sparsest sequences (seq 01 and seq 04, the latter only 271 frames), which the count weighted pool down-weights and the unweighted mean does not. Both figures are reported so neither hides the spread. Finally, the residual seq-00 classical tuning exposure is bounded: dropping the seq-00 fold lowers pooled recall only from 0.717 to 0.695 (a 0.022 delta), so the historical seq-00 pipeline tuning does not drive the aggregate.

Detection Accuracy Versus Range

Recall is not uniform in range, and the way it varies is not the naive "far = worse" monotone one might expect. Table 4.8 stratifies detection by the range of each cluster's centroid from the ego sensor, on a stride-20 sample of sequence 08 under the frozen configuration. Deviation: like the clustering benchmark, this table is per frame with the track filter off (striding breaks the tracker), so its pooled recall (0.671) is a per frame figure and does not restate the track filtered headline 0.732. Read the table as a range trend, not as a re-derivation of the headline.

Table 4.8: Detection stratified by range-to-ego on a stride-20 sample of the full sequence (frozen configuration, per-frame / track filter off). Mean IoU is over matched detections.

Range (m)	Precision	Recall	F_1	Mean IoU (matched)	TP / FP / FN
0–10	0.899	0.787	0.839	0.902	311 / 35 / 84
10–20	0.869	**0.862**	0.866	0.926	432 / 65 / 69
20–40	0.799	0.550	0.652	0.934	401 / 101 / 328
40+	0.512	0.196	0.284	0.943	22 / 21 / 90
Pooled	0.840	0.671	0.746	0.923	1166 / 222 / 571

The counter-intuitive result is the nearest bin recall dip: recall at 0 m to 10 m (0.787) is lower than at 10 m to 20 m (0.862), even though near cars carry far more points. This is not sparsity, it is over-segmentation. The split rate analysis shows that HDBSCAN splits the majority of the very near cars (a split rate of 53.5 % at 0 m to 10 m, falling to 3.5 % past 40 m): a large, close car dissolves into several clusters, none of which matches the whole instance, so it is counted as a miss. The nearest bin also has the lowest mean IoU of matched detections (0.902), consistent with fragments eroding the overlap even on the cars that do match. We develop this splitting mechanism, and why post-hoc fragment merging fails to recover it, in Section 4.2.3.

Beyond the peak the decline is ordinary sparsity: recall falls from 0.862 at 10 m to 20 m to 0.550 at 20 m to 40 m and 0.196 past 40 m, as cars thin to too few surviving points to cluster or to clear the 10-point eligibility rule (Section 4.1). Mean IoU of matched detections rises with range (0.902 → 0.943): the far cars that are detected at all are compact, single cluster instances, so their overlap is clean even as their number collapses. The earlier precision only, seq-00, pre-promotion range analysis is superseded by this on frozen config sequence-08 table and is retained only as a historical note.

4.2.2 Ablation Studies

Every ablation is produced under the frozen protocol of Section 4.1 unless a deviation is stated explicitly.

Stage decomposition (classifier and track filter): Table 4.9 isolates the two filtering stages that sit on top of geometric clustering. The geometric only row uses clustering plus the geometric size/shape filters with no learned classifier and no track filter (like-for-like under the promoted configuration); the full row is the headline.

Table 4.9: Stage ablation on the full sequence (frozen configuration): rows add one filtering stage at a time on top of geometric clustering. Geometric-only and the middle row are per-frame (no track filter); the full row uses the offline track filter. Mean IoU is over matched detections only.

Configuration	Precision	Recall	F_1	Mean IoU	TP / FP / FN
Geometric only	0.199	0.773	0.316	0.907	27 002 / 108 831 / 7920
+ classifier	0.846	0.676	0.751	0.922	23 600 / 4303 / 11 322
+ classifier, track filter	0.911	0.732	0.812	0.913	25 568 / 2513 / 9354

The decomposition tells a two step story. The classifier is the precision mechanism: geometric clustering under the promoted coarse voxel setting emits many mostly junk clusters (precision 0.199), and the per frame classifier removes about 104 000 of them across the sequence (FP 108 831 → 4303), lifting precision from 0.199 to 0.846. It pays for this with recall, being conservative, it discards borderline true clusters too, so recall falls from 0.773 to 0.676. The track filter then recovers that recall while improving precision again: linking detections across frames and accepting only consistent tracks lifts precision to 0.911 and recall to 0.732 (TP 23 600 → 25 568), because a car seen across many frames is kept even where a single frame classified it as unknown, while transient false positives fail the track length test. The net of both stages is still a recall trade: geometric only recall (0.773) exceeds the full pipeline recall (0.732), roughly 1400 true positives forgone for the ∼106 000-false positive removal. But that trade runs through a recall dip and a partial recovery, not a simple monotone loss. The system is deliberately tuned for precision, and the recall it forgoes here is small next to the recall it never had (Section 4.2.3). The pre-promotion geometric only figure (F_1 0.305) is superseded by this like-for-like 0.316 and kept only as a historical record.

Redundancy of synthetic pretraining (Stage A ablation): The classifier ships as a binary car/not-car network trained from scratch on mined real clusters (Stage B). A natural alternative is a two stage recipe in which a synthetic ShapeNet Stage A pretraining seeds Stage B; we tested whether that pretraining is necessary and found it is not, a negative result we report as a finding, because it changes the recommended recipe.

At the classifier level, on the sequence 08 validation clusters, the from scratch network matches the pretrained one: macro F_1 0.9285 from scratch versus 0.9225 pretrained (the two are within run-to-run noise, with scratch marginally ahead). At the pipeline level the two checkpoints are identical in F_1 to three decimals on the full sequence 08. We therefore drop Stage A from the production pipeline and keep it as ablation material.

The reason the synthetic prior is redundant, and cannot simply be added for safety, is a total, symmetric domain gap between synthetic and real point clusters. Table 4.10 gives the cross-domain transfer matrix: a model trained on one domain and evaluated on the other scores a car class F_1 of 0.000 in both off-diagonal directions, and fine-tuning on real data erases whatever the synthetic pretraining had learned. Given 420 000 real training clusters, the real data does essentially all the work; the synthetic prior neither helps nor, once fine-tuned, survives.

Table 4.10: Cross-domain classifier transfer (car-class F_1): each model trained on one domain and evaluated on the other's validation set; diagonal cells (italic) are in-domain.

	Evaluated on	
Trained on	Synthetic	Real
Synthetic (Stage A)	0.999	0.000
Real (Stage B)	0.000	0.885

Cost and benefit of HDBSCAN clustering: The clustering stage is HDBSCAN. Because it is by far the most expensive stage (Section 4.2.4), we benchmarked it against two cheaper density/connectivity clusterers, DBSCAN and Euclidean tolerance clustering, at the same voxel resolution and under an identical pipeline (Table 4.11). Deviation: this

benchmark is per frame with the track filter off, because it strides across the sequence and the tracker requires contiguous frames; the numbers are therefore comparable to each other but not to the track filtered headline.

Table 4.11: Clustering method benchmark at fixed voxel resolution (0.10 m), identical pipeline, per-frame / track filter off. Mean IoU is over matched detections; the timing column is the mean per-frame clustering time.

Seq	Method	Precision	Recall	F_1	Mean IoU	Clustering (mean ms)
00	HDBSCAN	0.972	**0.731**	**0.835**	0.968	306
	DBSCAN	0.975	0.637	0.771	0.964	54
	Euclidean	0.979	0.636	0.771	0.982	76
08	HDBSCAN	0.840	**0.671**	**0.746**	0.923	320
	DBSCAN	0.870	0.565	0.685	0.927	57
	Euclidean	0.869	0.576	0.693	0.950	75

The gap is entirely a recall gap, precision is within 0.030 across all methods on either sequence, while HDBSCAN's recall exceeds the alternatives by 9 to 11 points. An ε-sweep confirms no fixed radius closes it: the best fixed radius setting (Euclidean at $\varepsilon = 0.4$ m) reaches F_1 0.800 on sequence 00, still short of HDBSCAN's 0.835. HDBSCAN's variable density criterion recovers cars that a single radius cannot, which is exactly the recall margin that matters here. We therefore adopt nothing from the benchmark and keep HDBSCAN in production, paying its runtime cost knowingly, roughly 4 to 6 times the DBSCAN and Euclidean clustering time here; DBSCAN and Euclidean remain available as command line options for reproduction.

4.2.3 Result Analysis and Error Analysis

The detection operating point above established two facts: the pipeline is precision saturated (false positives are low and flat, about 0.6 per frame), and it is recall limited, missing roughly one car in four ($R = 0.732$ on sequence 08). This section explains why the recall is limited, and it also underpins the completion results (Section 4.3.1): the same clustering step that caps recall is the one that decides whether a detected car reaches completion as a clean single object cluster. We show that the shortfall is a clustering failure rather than a classification failure (Section 4.2.3) and root cause it to HDBSCAN splitting large, close cars into fragments (Section 4.2.3). We then report a search of repair strategies that all fail to generalise (Section 4.2.3), and describe the one intervention that did help, coarsening the clustering resolution, which forces a correction of an earlier, overstated "hard ceiling" claim (Section 4.2.3). The negative results are reported as findings: mapping which repairs fail, and why, is itself a finding worth reporting, and it is what justifies the residual limitation carried to Chapter 5. Unless stated otherwise, the recall denominator throughout is the one defined in Section 4.1, ground truth car instances with at least 10 points surviving preprocessing.

The Shortfall as a Clustering Failure

The recall ceiling is upstream of the learned classifier. The stage decomposition of Section 4.2.2 makes this direct: geometric clustering with the size/shape filters but no

learned classifier and no track filter already reaches recall 0.773, which is higher than the full pipeline's 0.732. Every learned stage that follows can only hold or reduce that recall: the classifier discards clusters, and the centroid tracker only re-labels and filters clusters that geometric clustering has already produced: it emits a detection only where a cluster already exists in that frame, and never interpolates one across a gap. Neither stage can invent a detection where clustering found nothing. Across the full sequence the two stages together trade roughly 1400 true positives (27 002 geometric only versus 25 568 full) for the removal of ∼106 000 false positives; this is a distinct trade from the +756 true positives that the resolution change of Section 4.2.3 later recovers. Whatever caps recall is therefore already present in the clusters that geometric clustering produces, before the classifier sees them.

The geometric filters are not the cap either. Ablation of the size/shape gates across the full sequence 08 shows that of the 903 901 HDBSCAN clusters, 127 654 (14.1 %) pass the filters, and 27 718 of the rejected clusters overlap a ground truth car, dominated by `min_volume` (73.9 % of the GT matching rejections) and `min_points` (12.7 %). The dominant gates are `min_volume` and `min_points`, the two that reject small clusters. This is consistent with the rejected clusters being the smaller sub-fragments of cars that HDBSCAN has already split (the dominant fragment passing while the splinters fail the size gates), which the split analysis of Section 4.2.3 then confirms is the prevailing failure mode. We state this as an inference rather than a direct measurement: the ablation records only that a rejected cluster overlaps some ground truth car, not whether that same car also retains a surviving cluster, so it establishes that the rejections are dominated by small fragments, not that every one belongs to an otherwise detected car. Either way the diagnosis points one stage further upstream, at the clustering itself.

Root Cause: HDBSCAN Fragmentation of Large, Close Cars

To quantify how clustering caps recall we measured, for every eligible ground truth car, how its points distribute across HDBSCAN clusters. Each car is assigned to exactly one of four states that partition the population: ok (one dominant cluster, under 30 % noise), split (points spread over two or more clusters), noisy (one cluster but at or above 30 % noise), or all noise. A separate, cross-cutting flag marks merged cars (a car whose dominant cluster is shared with another car). A merged car still has a dominant cluster, so it is not a fifth partition class, and is reported below as a memo count rather than a summed table row. Table 4.12 gives the result on the full sequence.

Table 4.12: Distribution of ground-truth cars across HDBSCAN clusters (full sequence, 34 057 eligible cars, pre-promotion 0.05 m clustering configuration). The four states partition the population; the merged row is a cross-cutting memo count, not summed.

	Seq 08 (34 057 cars)	
Status	Count	%
ok (single dominant cluster)	18769	55.1
split (2+ clusters)	15245	44.8
noisy / all noise	43	0.1
partition total	34057	100
merged (cross-cutting)	56	0.2

Three things follow. First, splitting, not merging, is the failure mode: 44.8 % of cars split, while merging is 0.2 %. The pipeline almost never fuses two cars into one detection; it repeatedly breaks one car into several. Second, the split is worst where cars are largest and nearest: the split rate falls monotonically with range, at the adopted 0.10 m resolution 53.5 % at 0 m to 10 m, 40.2 % at 10 m to 20 m, 22.3 % at 20 m to 40 m, and 3.5 % past 40 m. Large, close cars return many points, and HDBSCAN finds internal density variations within that dense return, the roof line against the body, the near side against the far side, and cuts the car along them. This is the mechanism behind the counter-intuitive near range recall dip reported in Section 4.2.1: the nearest bin has lower recall than the 10 m to 20 m bin despite carrying far more points, because a close car dissolves into fragments that no longer match the whole instance. Third, the cleanly clustered fraction is a floor on recall, and splitting is the dominant loss above it. The fraction of cars that form one dominant cluster (55.1 % at the pre-promotion configuration) sits below the recall the pipeline achieves there (0.710): the pipeline recovers essentially every cleanly clustered car and, in addition, a portion of the split cars whose dominant fragment survives matching and filtering, so recall clears the clean fraction rather than sitting at it. Splitting nonetheless remains the dominant loss mode, 44.8 % of cars split and the majority of those are still lost, because a large dominant point fraction (median 0.79) does not translate into a match when the fragment must also clear the size filters. This predicts the resolution fix of Section 4.2.3: coarsening the voxel converts split cars into clean ones, which should lift recall by raising the clean fraction and shrinking the reliance on recovering split fragments. It does so, raising the clean fraction from 55.1 % to 66.0 %, recall from 0.710 to 0.732, and narrowing the gap between clean fraction and recall from 15.9 to 7.2 points.

Figure 4.3 shows the mechanism directly on three sequence 08 frames. Missed cars (orange) appear as broken rows of points close to the ego vehicle, each row too small on its own to survive the size filters, while cleaner mid- and far range cars (green) are detected as single clusters.

Limits of the Repair Strategies

Given that splitting caps recall, the obvious responses fall into two families: change how clusters form (a larger minimum cluster size, a different clusterer, distance adaptive parameters, or pre-clustering point accumulation) or reassemble fragments after HDBSCAN has split them. We tried six in total; Table 4.13 collects the four with a reproducible quantitative outcome, and two further strategies (temporal accumulation and threshold lowering) are discussed in prose below. All were evaluated against the same pre-promotion sequence-00 baseline (full pipeline, classifier and track filter on; $F_1 = 0.844$), and the load bearing ones were re-checked on the held out sequence 08.

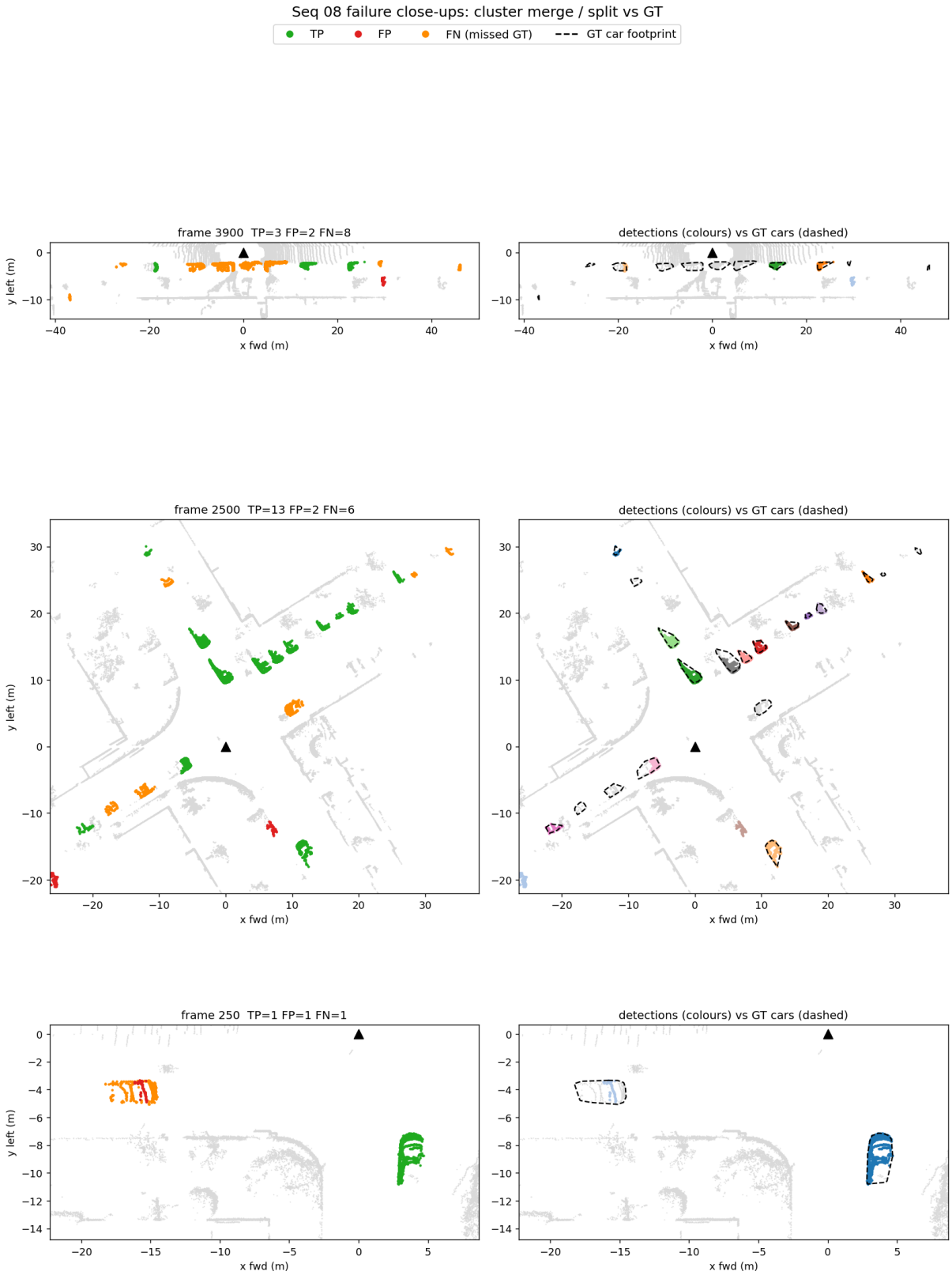


Figure 4.3: Failure close-ups: true positives green, false positives red, missed ground-truth cars orange, and dashed outlines the ground-truth footprints. The misses shown are fragmentation (split), not fusion of neighbouring cars.

Table 4.13: Recall repair strategies against the pre-promotion sequence-00 baseline (full pipeline, classifier and track filter on; $F_1 = 0.844$), with held-out sequence-08 F_1 where measured. This exploratory search predates the coarse-voxel configuration of Section 4.2.3; "Src" is the finding number, and two further strategies (temporal accumulation, threshold lowering) are discussed only in the text.

Strategy	Seq 00 P	Seq 00 R	Seq 00 F_1	Seq 08 F_1
Baseline (HDBSCAN, MCS 10)	0.984	0.739	**0.844**	**0.829**
BEV clustering (res 0.30 m)	0.993	0.640	0.779	,
Higher MCS 20	0.976	0.755	0.852	0.801
Fragment merge (1.5 m, 30 pt)	0.931	0.755	0.834	0.815
Adaptive HDBSCAN (30/15/10)	0.935	0.748	0.831	,

Each strategy fails for a structural reason, and the reasons are consistent with the diagnosis of Section 4.2.3:

- Higher minimum cluster size (MCS 20) is the most instructive. It is the only variant to improve F_1 on sequence 00 (0.844 → 0.852), by discouraging the small splinter clusters. But it does not generalise: on the held out sequence 08 it drops to F_1 0.801, below the 0.829 baseline, because a larger minimum also erases genuinely sparse distant cars. The sequence-00 gain was over-fitting.

- Post-clustering fragment merge (join clusters within 1.5 m) raises sequence-00 recall to 0.755 but loses precision (0.984 → 0.931), because the same proximity rule also fuses nearby non-car structure (walls, poles) into oversized clusters. On held out data the precision loss cancels the recall gain (F_1 0.815 < 0.829).

- Distance adaptive HDBSCAN (a larger minimum cluster size near the ego, smaller far away) introduces boundary artefacts at the ring transitions and is worse than the global setting.

- Bird's-eye-view clustering, a cheaper connected component alternative, collapses recall to 0.640: projecting to two dimensions merges objects that overlap in the ground plane but are separated in height, and morphological cleanup destroys sparse distant clusters.

- Temporal aggregation, accumulating object points over three consecutive frames before clustering, to densify sparse cars, collapses the pipeline (F_1 0.073, recall 0.039).[2] Accumulating points does not densify the existing clusters; it reshapes the clustering landscape entirely (about 134 000 accumulated points yield 645+ clusters versus ∼240 single frame, of which fewer pass the geometric filters, 17 versus 34).

- Lowering the cluster size thresholds to admit sparser groups produced no change in true positives at all: the binding constraint is cluster quality, not the count threshold, so relaxing the threshold merely admits more noise (`docs/session_history.md`).

[2]Source: `docs/session_history.md` (2026-06 exploratory session). The `temporal_window` feature was implemented and then fully reverted, so this result is on the pre-promotion baseline and was not re-run under the research freeze; it is reported here in prose, not in Table 4.13, for that reason, and its value is the mechanism below rather than the exact figure.

The failure modes are strategy specific, over-fitting, precision loss, ring artefacts, projection ambiguity, a reshaped clustering landscape, but they share a limit. The two strategies that reassemble after the split (fragment merge, and to a lesser extent the proximity-based part of the adaptive scheme) run into the fact that density-based clustering has no notion of objectness: once a dense car is partitioned along an internal density gap, no proximity- or size-based rule can reliably tell "rejoin these two fragments of one car" apart from "do not join these two adjacent cars." The strategies that instead change how clusters form, a larger minimum size, a different clusterer, per ring parameters, pre-clustering accumulation, avoid the reassembly trap but hit the same wall from the other side: any single global density criterion that is loose enough to keep a split car whole is also loose enough to fuse a close pair, so a sequence-00 gain becomes a held out loss. The one lever that helped, changing the clustering resolution (Section 4.2.3), is also a formation change; it works not by escaping this limit but by happening to sit at a resolution that closes intra-car gaps before it starts closing inter-car ones. Initially, the recall appeared constrained to a hard ceiling of about 0.74. Given only the evidence in Table 4.13, that conclusion was reasonable. It was nonetheless mistaken: it conflated a limit on these repair strategies with a limit on the approach itself, as the next section shows.

Clustering Resolution and the Corrected Ceiling

The intervention that helped was not a repair at all but a change to the clustering resolution: re-voxelising the object cloud to a 0.10 m grid before running HDBSCAN, rather than clustering the full resolution 0.05 m cloud (adopted in the runtime optimisation of Section 4.2.4). A coarser grid closes the small intra-car density gaps that HDBSCAN was cutting along, so fewer splits form in the first place. It is a formation change like the failed ones in Section 4.2.3, but it lands on the favourable side of the trade described there, coarse enough to close intra-car gaps, still fine enough not to fuse neighbouring cars.

Table 4.14: Effect of coarsening the clustering voxel from 0.05 m to 0.10 m on the split rate (full sequence, 34 057 ground-truth cars).

	Full res (0.05 m)	Coarse (0.10 m)
clustered "ok"	18769 (55.1%)	22487 (66.0%)
split	15245 (44.8%)	11480 (33.7%)
merged (cross-cutting)	56 (0.2%)	284 (0.8%)
mean clusters per split car	4.7	3.0

Table 4.14 shows the split rate falling by 11.1 points (44.8 % → 33.7 %). This is a trade, not a free lunch: the coarser grid also fuses some adjacent cars, so the merged count rises from 56 to 284 (0.2 % → 0.8 %). The gain still dominates, roughly 3700 cars move from split to clean while about 230 newly merge, but the merge cost is real and bounds how far coarsening can be pushed. At the pipeline level, on the full sequence 08, this lifts recall from 0.710 to 0.732 (+756 true positives) with precision essentially unchanged (0.910 → 0.911) and F_1 from 0.798 to 0.812. The same coarsening also reduces clustering runtime, which is why it was adopted; its detection benefit is the subject here and its runtime co-benefit is reported in Section 4.2.4.

This result refines the earlier ceiling claim. The "∼0.74 hard limit" was partly a resolution artefact, not a fundamental property of the approach: the strategies of Sec-

tion 4.2.3 genuinely do fail, but none of them tried this particular resolution setting, and changing it recovered part of the loss. The standing recommendation is therefore `cluster_voxel_size` = 0.10 m in production, not "accept 0.74."

A smaller structural limit does remain, and it is architectural. Coarsening the voxel further would eventually start merging adjacent cars, trading the split problem for a merge problem; and the clustering method benchmark of Section 4.2.2 shows that no fixed radius density clusterer, at any tested radius, reaches even HDBSCAN's recall. The ceiling that remains is the one imposed by density-based clustering having no model of what a car is: it groups points by proximity and density alone, so it cannot be pushed to the recall of a learned object detector by tuning. Reaching that level would require replacing the clustering stage with a learned detector, which is outside the scope of this thesis (Chapter 5). This analysis locates the ceiling precisely. It lies in the clustering stage; it is splitting rather than classification or merging; and it is lower than first claimed. The analysis also maps which repairs cannot move it and which one can.

Recall Analysis: Summary

The recall shortfall is clustering bound, not classifier bound: the classifier and track filter sit on top of a recall ceiling that geometric clustering already imposes (Section 4.2.3). The dominant mechanism is HDBSCAN splitting large, close cars into fragments, 33.7 % of cars at the adopted resolution, worst at short range, while merging is negligible, and the cleanly clustered fraction sits just below the achieved recall, so most split cars are lost (Section 4.2.3). A search of six repair strategies, higher minimum cluster size, fragment merging, adaptive and BEV clustering, temporal accumulation, and threshold lowering, all fail to generalise, for a shared reason: density-based clustering has no notion of objectness, so any criterion loose enough to keep a split car whole also fuses adjacent cars, and reassembling fragments after the split faces the same ambiguity (Section 4.2.3). The one effective lever was clustering resolution: a coarser pre-clustering voxel prevents splits from forming, lifting recall 0.710 → 0.732 and correcting the earlier overstated "hard ceiling" (Section 4.2.3). The residual limit is architectural, density clustering has no notion of objectness, and is carried to the limitations and future work discussion of Chapter 5.

4.2.4 Runtime Evaluation

Table 4.15 breaks the per frame cost down by stage, measured over the full sequence 08 under the final (promoted) configuration (4068 timed frames, RTX 3070 Ti / Ryzen 7 7800X3D; per stage figures from `timing_seq08_full_n4068_combined.json`). The breakdown shows where the time goes. The two classical geometric stages, ground plane RANSAC and, above all, HDBSCAN clustering, dominate, together taking roughly 68 % of the frame. Adding preprocessing and the geometric filter, the classical CPU stages account for about 94 %, leaving the single learned component, the classifier, at only ∼5.4 % (the centroid/Hungarian tracker is a classical assignment step, not learned). The classifier is 33.7 ms for the ∼45 clusters in a frame (∼0.75 ms each) and the tracker is negligible.

Table 4.15: Per-frame runtime by stage on the full sequence under the final (promoted) configuration (4068 timed frames; mean ms). Percentages are of the 623.5 ms total.

Stage	Mean (ms)	Share
Data load	0.7	0.1%
Preprocessing (z filter, denoise, voxel)	125.9	20.2%
Ground removal (RANSAC)	56.2	9.0%
Clustering (HDBSCAN)	370.4	59.4%
Geometric filter	36.2	5.8%
Classification	33.7	5.4%
Tracking	0.3	0.1%
Total	623.5	100% (≈1.6 fps)

The promoted configuration is itself the product of a runtime optimisation that targeted the dominant classical stages directly: reducing the RANSAC iteration count and coarsening the clustering voxel to 0.10 m cut the per frame total by roughly a third, from a pre-optimisation 921.3 ms to 623.5 ms on the full sequence (650.6 ms on the stride-20 sample). The coarser voxel also improved detection, so it is the promoted configuration used throughout this chapter. A sub-400 ms target was shown to be unreachable through the authorised optimisation tiers: the post-ground object cloud is intrinsically sparse, so coarser voxelisation compresses it little and the Python HDBSCAN retains a floor of ∼185 ms. The achieved operating point was accepted, real-time operation being out of scope (user decision, 2026-07-23). Point completion, when it runs, adds ∼13 ms per completed car on top of detection and is not part of the detection budget above.

4.3 Completion Evaluation

This section evaluates the completion half of the pipeline: the completion results across synthetic fidelity, real data plausibility, and reference free fidelity (Section 4.3.1), followed by the ablations that isolate each completion choice (Section 4.3.2).

4.3.1 Completion Results

The completion stage is evaluated on the two fronts the problem allows. On synthetic shapes a complete ground truth exists, so the network can be scored directly with Chamfer distance and F-score; this establishes that the network works in distribution (Section 4.3.1). On real LiDAR no complete ground truth shape exists (Section 2.1.4), so the completion is judged two ways that need no self-made reference: an independent shape classifier tests whether the completed real clouds read as cars (Section 4.3.1), the favorable real data signal; and the established reference free fidelity metrics are reported with the caveat that they reward under-completion (Section 4.3.1). The scope of the resulting claim is stated plainly in Section 4.3.1.

Synthetic Fidelity: The Network in Distribution

Trained on the KITTI-like synthetic partials of Section 3.3.3 (input 256 points, target the dense mesh sample), the completion network (`pcn_kitti_best.pth`) reaches a best

validation Chamfer loss of 0.1246. On a held out synthetic set, where the complete CAD shape is the ground truth, it produces clean cars rather than blobs: Chamfer distance 0.16 m and F-score@0.1 m 0.76. These are established metrics against an established ground truth (ShapeNet), and they confirm the network has learned the partial-to-complete mapping in distribution. They say nothing on their own about real LiDAR, which is what the next two subsections address.

Real Data Plausibility: An Independent Classifier

On real cars there is no complete shape to score against, so we adopt the plausibility protocol of Chen et al. [26]: an independent shape classifier, trained on complete shape data disjoint from the completion network's training set, judges whether each completed cloud reads as a car. The judge is a PointNet++ (single scale grouping) trained from scratch on ModelNet40, whose CAD models are drawn from a different repository than the ShapeNet cars PCN was trained on, so a car accepted by the judge is not an artefact of shared training data. Chen et al. use a ModelNet40-trained judge for exactly this reason. The judge reaches 0.9214 test accuracy on ModelNet40 (above a pre-registered 0.88 usability gate) and is trained without rotation augmentation, so it is orientation sensitive. Every cloud is therefore harmonised into the judge's canonical frame (width, up, length), resampled to 1024 points, and unit sphere normalised, before scoring. The experiment and its hypotheses were pre-registered before any scoring run.

We score 2822 completed car tracks across all eleven labelled sequences, each paired with its own raw single frame partial, and report the car acceptance rate (the fraction the judge labels car) and the mean car probability. As a shape quality upper bound we also score 140 static accumulation clouds: dense multi-frame accumulations of well-observed static cars, which are both dense and correctly shaped, the ceiling a completion could aspire to. Table 4.16 reports the three arms.

Table 4.16: Independent-classifier plausibility on real LiDAR cars (pooled over all eleven labelled sequences; $N = 2822$ paired raw/completed tracks, plus a separate $N = 140$ static-accumulation ceiling of dense, correctly shaped cars). Acceptance is the judge's car argmax rate; car probability its mean car posterior.

Arm	Acceptance ↑	Mean car prob ↑
Raw partial	0.253	0.219
Completed	**0.785**	**0.745**
Static accumulation ceiling	0.807	0.728

Completion raises car acceptance from 0.253 on the raw partials to 0.785, and mean car probability from 0.219 to 0.745. The paired gain is decisive: on a McNemar test of the paired argmax decisions, completion flips 1615 rejected partials to accepted against only 115 in the reverse direction ($p \approx 0$), and the paired car probability improvement is significant on a Wilcoxon signed rank test ($p \approx 0$). This is the primary favorable real data completion result: an external judge, with no stake in the completion network's training data, reads the completed clouds as cars far more often than the partials it started from.

Completion reaches the shape quality ceiling: The raw→completed gain could in principle be a density effect: the completion is denser than the partial, and the judge

might reward density alone. The static accumulation ceiling controls for this, because it is both dense and correctly shaped. The completed clouds reach it. On the static subset ($N = 2220$) the completed mean car probability is 0.730, statistically at the ceiling's 0.728 (the Mann-Whitney $p = 6.8 \times 10^{-7}$ reflects distribution shape at large N, with the completed central tendency at or above the ceiling). Completed acceptance 0.770 recovers about 96% of the raw→ceiling acceptance gap. A completion that merely produced denser blobs would sit well below a ceiling built from correctly shaped cars; that it reaches the ceiling is evidence the completed shapes are genuinely car-like, not just dense.

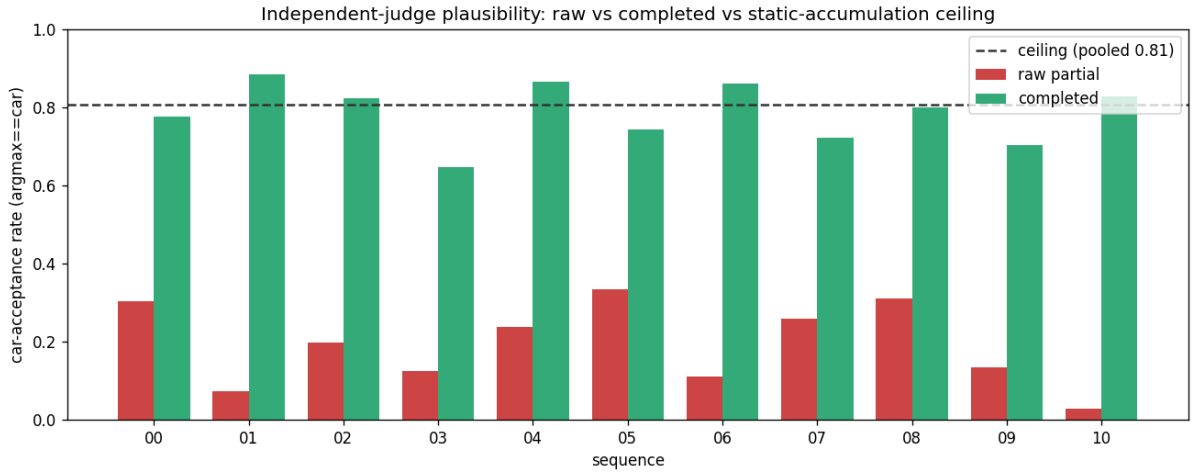


Figure 4.4: Car acceptance of the independent judge, per sequence: raw single-frame partial (red) vs. PCN completion (green), with the pooled static-accumulation ceiling (dashed).

The gain holds on every sequence: The completion constants were developed on sequence 08, so a fair test is whether the plausibility gain appears on sequences that took no part in that development. Figure 4.4 and Table 4.17 show it does: completed acceptance ranges from 0.646 to 0.885 across the eleven sequences, several times the raw partial rate on each, and sequence 08 (0.311 → 0.799) is unremarkable among them. The completion network was trained only on synthetic data and never saw any real sequence, so this consistency is a direct generalisation result.

Table 4.17: Car acceptance by sequence, raw partial vs. completed, over all eleven labelled sequences (N = paired tracks per sequence).

Seq	00	01	02	03	04	05	06	07	08	09	10
N	926	96	329	48	59	258	164	213	518	141	70
Raw	0.30	0.07	0.20	0.13	0.24	0.33	0.11	0.26	0.31	0.13	0.03
Compl.	0.78	0.89	0.82	0.65	0.86	0.74	0.86	0.72	0.80	0.70	0.83

Moving cars complete as plausibly as static ones: Because completion runs on a single reference frame's cluster rather than on accumulated points, there is no motion smear in its input, so motion is not expected to matter. Labelling each track static or moving by the net displacement of its centroid confirms it: the moving subset ($N = 602$) reaches completed acceptance 0.837 and car probability 0.801, if anything slightly above

the static subset. The static only ceiling comparison above therefore does not hide a mover failure mode.

Heading ambiguity is not a confound: The harmonisation fixes each cloud's up axis and length axis but leaves front-versus-back ambiguous (the footprint fit does not resolve heading sign). Re-scoring every completed cloud with an added 180° heading flip changes acceptance by only 0.009, so the residual heading ambiguity does not affect the result.

Reference Free Fidelity and Its Caveat

The field's established reference free metrics score a completion by how closely it reproduces its own observed input: the unidirectional Chamfer and Hausdorff distances (UCD, UHD; Section 2.1.4), applied to real KITTI cars by Chen et al. [26] and P2C [27]. Computed on the seq-08 cars (input $\rightarrow$ cloud direction, per car medians), they behave exactly as their definition dictates (Table 4.18): the raw partial is a trivial optimum at 0, because it is its own input, while the completion is scored worst, penalised for every point it adds beyond the observed side.

Table 4.18: Reference-free fidelity on real cars (39 cars, per-car medians, input $\rightarrow$ cloud direction; lower means closer reproduction of the observed input).

Cloud	UCD (m) $\downarrow$	$\text{UHD}_{\max}$ (m) $\downarrow$	UHD_{p95} (m) $\downarrow$
Raw partial	0.000	0.000	0.000
Mirrored	0.000	0.000	0.000
Completed	0.090	0.286	0.183

This is not a negative result about the completion; it is a property of the metrics. An input fidelity measure is minimised by the cloud that adds nothing, so it structurally cannot reward recovering unobserved surface. This is precisely why the unpaired completion literature pairs UCD/UHD with a separate plausibility term [26], and why this thesis reports fidelity honestly here but rests its favorable real data claim on the plausibility of Section 4.3.1.

Scope of the Completion Claim

The completion evidence is deliberately scoped. The network works in distribution on synthetic shapes (Chamfer 0.16 m, F@0.1 m 0.76), but synthetic performance is not a real data claim. On real LiDAR the favorable evidence is the independent classifier plausibility: completion lifts car acceptance from 0.253 to 0.785, reaching the static accumulation shape ceiling, consistently across all eleven sequences and for moving as well as static cars. The reference free fidelity metrics do not credit the completion, and are reported as such rather than omitted. The claim the thesis makes is therefore this and no more: on real automotive LiDAR, learned completion produces shapes an independent classifier accepts as cars far more often than the raw partials, and as often as dense correctly shaped references, while established fidelity metrics, which reward reproducing the input, do not credit it. A remaining limitation of the production path is the per car length estimate's fixed prior fallback on tracks with fewer than five gate passed frames (Section 3.3.6). On the seq-08 output this fallback is used by a non-trivial fraction of tracks, and is revisited in the discussion.

4.3.2 Ablation Studies

These ablations isolate the effect of each completion pipeline choice; they vary one component of the completion path (Section 3.3) and re-run the completion, with the completion network itself unchanged.

Inference normalisation: PCN is trained in a canonical car frame, so a real partial must be mapped into that frame before completion (Section 3.3.4). The importance of this step is the semantics map lesson of Section 3.3.8 made concrete: with the canonical frame normalisation, the completed clouds are recognisably car-shaped and correctly oriented. With a naive alternative, aligning the partial to a 3D PCA frame and normalising by its own centroid and radius, the same checkpoint produces mis-oriented, mis-scaled blobs, because the 3D PCA picks an arbitrary principal axis rather than gravity. The blob-versus-car difference is entirely a normalisation effect, not a network one, since the checkpoint is identical.

Input quality gate: The L-shape gate decides whether a detected car reaches completion as a clean single object cluster (Section 3.3.5). It rejects HDBSCAN fragments and merges, which would otherwise feed the completion network an input that is not one whole car and complete into an implausible shape. On seq 08 every implausible completion originated from a fragment or merge input, so the gate is what keeps the production completions clean; it carries a measured cost, dropping a fraction of otherwise usable cars as false rejections.

Length geometry: A partial truncated at the occluded far end completes short, so the completion tends to under-extend length (Section 3.3.6). The production path corrects this with a per car length estimate rather than a single fixed prior, because a population median prior over-extends genuinely compact cars while under-extending long ones. The per car estimate is the frozen production choice; a residual far end under-extension on the longest cars remains, characterised rather than eliminated, and is carried to the discussion.

Completion architecture (PCN vs. PoinTr): Swapping the completion network holds the rest of the pipeline fixed. On held out synthetic cars PoinTr's edge over PCN is small (Chamfer $0.161 \rightarrow 0.153$ m), a difference too small to justify the heavier model, and the residual real data gap is a centroid estimation limit that is architecture independent. PCN is therefore kept off the shelf; the completion result is set by input quality and geometry, not by the choice of network.

Chapter 5

Conclusion and Future Work

The preceding chapters reported results; this one interprets them and closes the thesis. It first draws the conclusions, answering the two research questions and restating the contributions (Section 5.1). It then gathers what the evidence supports and how far each claim reaches, together with every limitation the work carries, into a single section (Section 5.2): the validity of the evaluation, the scope of the completion claims, the ground truth assumptions, and the architectural and operational limits. Finally, it turns the most consequential boundaries into future work (Section 5.3). Every weakness named here is measured elsewhere in the document and cross-referenced to the finding that quantifies it: a limitation the thesis names, quantifies, and bounds is defended; one a committee surfaces first is not.

5.1 Conclusion

Chapter 1 posed two research questions, one for each half of the framework; each is answered by the evaluation of Chapter 4 and restated here in the form asked, with its reach discussed in Section 5.2.1.

RQ1, can a modular pipeline requiring zero additional manual annotation detect and complete occluded vehicles on real LiDAR? Yes. The framework detects cars with a chain of training free classical stages and two small learned models: the classifier trained on supervision auto-mined from existing semantic annotations, and the completion network on synthetic shapes. It reaches precision 0.888, recall 0.717, and F_1 0.793 pooled under leave-one-sequence-out cross-validation across all eleven labelled sequences, and completes each detected car with a PCN network. Every stage is separately inspectable, which is what lets the recall shortfall be root caused to a single stage rather than left opaque.

RQ2, does learned completion produce car-plausible shapes on real LiDAR, and how does it fare on established fidelity metrics? On real cars an independent shape classifier, trained on external complete shape data, accepts the completed clouds as cars far more often than the raw partials (car acceptance 0.253 $\rightarrow$ 0.785, paired McNemar and Wilcoxon $p \approx 0$) and about as often as a dense static accumulation shape ceiling (0.807). This holds consistently across all eleven sequences and for moving as well as static cars, so completion produces genuinely car-plausible shapes rather than denser blobs. The established reference free fidelity metrics (UCD, UHD) do not credit the completion: they

reward reproducing the observed input, so the raw partial is their trivial optimum. The favorable real data claim therefore rests on plausibility, with fidelity reported honestly as unfavorable.

Diagnostic findings: Two findings emerged along the way (Section 4.2.3, Chapter 4): recall is clustering bound, not classifier bound, HDBSCAN splits 33.7 % of cars and a chain of repair strategies fails to generalise, and synthetic pretraining is redundant once real clusters are plentiful, the sim-to-real gap being total and symmetric.

Contributions: The thesis makes two contributions. The first, and main, is a modular detection and completion framework for occluded vehicles on real LiDAR: an end-to-end, separately inspectable pipeline that integrates established components (RANSAC, HDBSCAN, PointNet, PCN) into a coherent, reproducible system, evaluated end to end. The second is auto-mined supervision and synthetic completion, an approach requiring zero additional manual annotation. It is built on SemanticKITTI (real scans for detection, mining, and evaluation) and ShapeNet (synthetic complete shapes for completion training), and its real data completion quality is assessed with an independent classifier so that no self-made reference is involved. Two diagnostic findings sit alongside them, the clustering bound recall account and the redundancy of synthetic pretraining, together with the reproducibility infrastructure and documented negative result lines.

5.2 Limitations and Discussion

5.2.1 Established Results and Their Reach

The framework rests on two halves, each with its own evidence. The detection half is the broadest result: an interpretable clustering front end, requiring zero additional manual annotation, reaching precision 0.888, recall 0.717, F_1 0.793 pooled under leave-one-sequence-out cross-validation across all eleven labelled sequences (Section 4.2.1; 0.911 / 0.732 / 0.812 at the sequence-08 operating point, mean IoU of matched detections 0.913; recall denominator: car instances with at least 10 points surviving preprocessing, Section 4.1), on modest hardware and without state-of-the-art accuracy, which the supervised networks of Section 2.1.5 reach through dense per point supervision, whereas this pipeline requires zero additional manual annotation. Its recall shortfall is root caused, not mysterious, HDBSCAN splitting large, close cars into fragments (Section 4.2.3).

The completion half's favorable real data evidence is the independent classifier plausibility (Section 4.3.1): completion lifts car acceptance from 0.253 on the raw partials to 0.785, reaching the dense static accumulation shape ceiling (0.807), consistently across all eleven sequences and for moving as well as static cars, with the paired gain significant on McNemar and Wilcoxon tests ($p \approx 0$). This is an established protocol (Chen et al. [26]) with an external judge disjoint from the completion network's training data, so it is not circular. In distribution, on synthetic shapes with a complete ground truth, the network scores Chamfer 0.16 m and F@0.1 m 0.76. An important limitation is that the established reference free fidelity metrics (UCD, UHD; Section 4.3.1) reward reproducing the observed input, so they cannot credit the completion, and the favorable claim rests on plausibility alone.

These reaches vary. The detection operating point is cross-validated across all eleven sequences (Section 4.2.1), and the plausibility gain likewise appears on every sequence,

so both halves transfer beyond the tuning sequence. The recall diagnosis is an inductive result from the labelled sequences, mechanistic but not a proven invariant, and the tuned completion constants (the length prior, the estimate quantile, the geometric thresholds) are sequence derived and should be re-checked on a new sensor. For practice, three readings follow. First, the interpretable detector, requiring zero additional manual annotation, is a viable choice where labelling budget or inspectability outweighs peak recall, provided the domain is a dense scene. Second, independent classifier plausibility is a reusable, non-circular way to assess real LiDAR completion where no complete ground truth exists. Third, because input quality, not model choice, governs whether completion is usable, effort is better spent on segmentation than on swapping architectures. These reaches are bounded by the limitations set out next.

5.2.2 Validity of the Evaluation

Detection is cross-validated, and the selection exposure is precision only. SemanticKITTI labels only sequences 0 to 10, so no labelled sequence can be reserved as a conventional held out detection test set, and the final classifier was historically selected on sequence 08. Leave-one-sequence-out cross-validation (Section 4.2.1) bounds the exposure: the fold-08 classifier matches its recall (0.736 versus 0.732) and loses only about two precision points (0.888 versus 0.911). The selection therefore touched precision, not the clustering imposed recall ceiling (0.773) that accounts for most missed cars. The completion evaluation is guarded on two fronts. It is non-circular: the plausibility judge is trained on ModelNet40, a shape repository disjoint from the ShapeNet cars the completion network learned from, so an accepted car is not an artefact of shared training data, and the completion network never saw a real LiDAR sequence. And it generalises: the acceptance gain is not a single held out replication but appears on all eleven labelled sequences (Section 4.3.1), with the tuning sequence 08 unremarkable among them. One asymmetry remains: the favorable evidence is plausibility alone; the established fidelity metrics do not corroborate it, because they reward under-completion, and synthetic fidelity is not a real data claim.

5.2.3 Scope of the Completion Claims

The completion claim is deliberately bounded. The favorable real data evidence is a single established instrument, independent classifier plausibility, and it measures whether a completed cloud reads as a car, not metric surface accuracy against a complete shape, which no real LiDAR reference can supply. The established reference free fidelity metrics do not corroborate it, so the claim rests on plausibility alone and must not be read as a surface reconstruction accuracy claim. Plausibility scores only clean, gate passed clusters, and whether a car reaches completion as one is decided upstream by the clustering that caps recall: the L-shape input gate rejects fragments and merges that would otherwise complete implausibly (Section 4.3.2), whereas swapping PCN for PoinTr changes the result not at all, so widening completion's reach is a segmentation problem, not a completion architecture one.

Within that scope, two residuals bound the completed shape. The per car length estimate falls back to the 4.14 m population prior on tracks with fewer than five gate passed frames, a non-trivial fraction of sequence-08 tracks, reintroducing some compact overshoot there, on the sparsest, most distant cars where it matters least. Even with an

unbiased estimate the completed cloud stays slightly short at the far end by an amount that scales with car length: a pre-registered fix removed this on normal and long cars but over-extended compacts, so the compensation is irreducibly length dependent and the far end remains the weakest covered region. Both residuals compound the model's synthetic only training, the source of the evaluation's independence, but also a synthetic-to-real gap (Section 2.1.4) behind the residual flatness of the sparsest real completions.

5.2.4 Ground Truth and Dataset Assumptions

Two assumptions underlie the ground truth. Recall is reported against eligible cars, those with at least 10 points surviving preprocessing, which excludes 25.5 % of annotated cars too sparse or distant to reach the front end. Recall against all annotated cars is therefore approximately 0.55, against the 0.732 eligible figure (both reported; the two "≥10-point" rules are distinct). And all counts and matches assume SemanticKITTI's labels are correct; we did not audit them, and mislabelled cars would propagate into the detection metrics and the mined classifier labels. The completion evaluation avoids a self-made reference entirely: its plausibility judge is scored against ModelNet40's own labels, and the raw-versus-completed comparison is paired on identical inputs, so no accumulated pseudo-ground truth enters the completion claim.

5.2.5 Architectural and Operational Limits

The remaining limits are architectural or operational. The recall ceiling is architectural, not a tuning failure (Section 4.2.3): coarsening the cluster voxel to 0.10 m recovered part of the loss (0.710 → 0.732), but coarsening further fuses adjacent cars, and no fixed radius density clusterer reaches HDBSCAN's recall at any tested radius. The ceiling (0.773 before any learned stage, splitting 33.7 % of cars) is therefore the measured price of the interpretable, zero-additional-annotation design the thesis exists to evaluate. Lifting it means a learned detector, a different system. The operational domain is dense scenes: recall falls to 0.336 on the sparse highway sequence 01 against 0.758 on dense urban sequence 00. This is mostly because highway cars are distant and recall collapses with range (0.196 beyond 40 m), and partly because the front end was tuned on dense data. The failure is nonetheless a conservative miss, not a false alarm (sequence 01 precision 0.951). The system is offline by design, running at about 623.5 ms per frame (Section 4.2.4) with the ≤ 400 ms target unreachable through the authorised tiers, and real-time operation is out of scope. Finally, the centroid tracker feeds the per car length estimate and drives its fixed prior fallback but is never scored as a tracker (no MOTA or IDF1), so its contribution to that fallback is unmeasured.

5.3 Future Work

The limitations above named the boundaries of the present results. Three of them are the natural next steps, each already scoped by a finding rather than left open.

A real-time variant: The pipeline runs offline at roughly 623.5 ms per scan (Section 4.2.4), dominated by the classical CPU stages. The direct speed path is an implementation one: the residual ∼185 ms attributed to clustering is the cost of the Python HDBSCAN, not the HDBSCAN algorithm, so a native or GPU implementation would

attack that floor without changing what is computed. Swapping in a faster fixed radius clusterer is not a free substitution: the benchmark shows none reaches HDBSCAN's recall at any tested radius. A real-time variant must therefore preserve the HDBSCAN algorithm via a faster implementation, or replace the front end with a learned detector, rather than trade measured recall for speed.

Real data fine-tuning for completion: The completion model was trained on synthetic KITTI-like partials only, which is at once the source of the evaluation's independence and a ceiling on real data crispness: on the sparsest, most one-sided inputs the completions stay flat, which traces to the remaining synthetic-to-real domain gap. Closing it would call for real data fine-tuning, which is itself a documented negative result line in this project; it is a contingency direction, to be attempted only with the leakage controls that make real data supervision safe.

Downstream detection utility: The plausibility result establishes that completed clouds read as cars, but not yet that completion improves a downstream perception task. The field's other real data payoff for completion is a measured improvement in 3D detection precision from feeding completed clouds to a detector (Ren et al. [28]). Wiring the completed cars into an oriented box estimator and measuring whether detection improves would turn the plausibility signal into a task level one, and is the natural next validation. It is deferred here as an integration task, not a change to the completion method.

None of the thesis's results rests on a self-made evaluation instrument. The main contribution is the modular framework itself. Each empirical claim, that a modular pipeline requiring zero additional manual annotation detects and completes occluded vehicles on real LiDAR, that its completions read as cars to an independent classifier, that recall is clustering bound, and that synthetic pretraining is redundant at scale, is defended by cross-validation, by an established external judge, or by explanation, and reported as it fell. That is the standard the work set for itself, and the standard on which it asks to be judged.

Bibliography

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.

[2] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall, "Semantickitti: A dataset for semantic scene understanding of lidar sequences," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

[3] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.

[4] C. R. Qi, L. Yi, H. Su, and L. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/d8bf84be3800d12f74d8b05e9b89836f-Paper.pdf

[5] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and A. Markham, "Randla-net: Efficient semantic segmentation of large-scale point clouds," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[6] B. Wu, A. Wan, X. Yue, and K. Keutzer, "Squeezeseg: Convolutional neural nets with recurrent crf for real-time road-object segmentation from 3d lidar point cloud," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE Press, 2018, p. 1887–1893. [Online]. Available: https://doi.org/10.1109/ICRA.2018.8462926

[7] A. Milioto, I. Vizzo, J. Behley, and C. Stachniss, "Rangenet++: Fast and accurate lidar semantic segmentation," in *2019 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2019, pp. 4213–4220.

[8] X. Zhu, H. Zhou, T. Wang, F. Hong, Y. Ma, W. Li, H. Li, and D. Lin, "Cylindrical and asymmetrical 3d convolution networks for lidar segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[9] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "Pointpillars: Fast encoders for object detection from point clouds," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[10] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3d object detection and tracking," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[11] Z. Zhou, Y. Zhang, and H. Foroosh, "Panoptic-polarnet: Proposal-free lidar point cloud panoptic segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[12] F. Hong, H. Zhou, X. Zhu, H. Li, and Z. Liu, "Lidar-based panoptic segmentation via dynamic shifting network," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[13] N.-Y. Lim and T.-H. Park, "Long-range lidar vehicle detection through clustering and classification for autonomous racing," *IEEE Access*, vol. 13, 2025.

[14] W. Yuan, T. Khot, D. Held, C. Mertz, and M. Hebert, "Pcn: Point completion network," 2019. [Online]. Available: https://arxiv.org/abs/1808.00671

[15] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu, "Shapenet: An information-rich 3d model repository," 2015. [Online]. Available: https://arxiv.org/abs/1512.03012

[16] J. Behley, A. Milioto, and C. Stachniss, "A benchmark for lidar-based panoptic segmentation based on kitti," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021.

[17] S. Gasperini, M.-A. N. Mahani, A. Marcos-Ramiro, N. Navab, and F. Tombari, "Panoster: End-to-end panoptic segmentation of lidar point clouds," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3216–3223, 2021.

[18] K. Sirohi, R. Mohan, D. Büscher, W. Burgard, and A. Valada, "Efficientlps: Efficient lidar panoptic segmentation," *IEEE Transactions on Robotics*, vol. 38, no. 3, 2022.

[19] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 1996, pp. 226–231.

[20] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Advances in Knowledge Discovery and Data Mining (PAKDD)*. Springer, 2013, pp. 160–172.

[21] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering," *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017.

[22] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[23] X. Yu, Y. Rao, Z. Wang, Z. Liu, J. Lu, and J. Zhou, "Pointr: Diverse point cloud completion with geometry-aware transformers," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 12 498–12 507.

[24] P. Xiang, X. Wen, Y.-S. Liu, Y.-P. Cao, P. Wan, W. Zheng, and Z. Han, "Snowflakenet: Point cloud completion by snowflake point deconvolution with skip-transformer," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 5499–5509.

[25] H. Zhou, Y. Cao, W. Chu, J. Zhu, T. Lu, Y. Tai, and C. Wang, "Seedformer: Patch seeds based point cloud completion with upsample transformer," in *European conference on computer vision*. Springer, 2022, pp. 416–432.

[26] X. Chen, B. Chen, and N. J. Mitra, "Unpaired point cloud completion on real scans using adversarial training," in *International Conference on Learning Representations (ICLR)*, 2020.

[27] R. Cui, S. Qiu, S. Anwar, J. Liu, C. Xing, J. Zhang, and N. Barnes, "P2C: Self-supervised point cloud completion from single partial clouds," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[28] Y. Ren, P. Cong, X. Zhu, and Y. Ma, "Self-supervised point cloud completion on real traffic scenes via scene-concerned bottom-up mechanism," 2022. [Online]. Available: https://arxiv.org/abs/2203.10569

[29] H. Fan, H. Su, and L. J. Guibas, "A point set generation network for 3D object reconstruction from a single image," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[30] X. Weng, J. Wang, D. Held, and K. Kitani, "3d multi-object tracking: A baseline and new evaluation metrics," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[31] X. Zhang, W. Xu, C. Dong, and J. M. Dolan, "Efficient L-shape fitting for vehicle detection using laser scanners," in *2017 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2017, pp. 54–59.